

Vertex Sampling for Backdoor Search in Binary Linear Programs

20TH LEARNING AND INTELLIGENT OPTIMIZATION CONFERENCE

Chang Liu, Elias B. Khalil

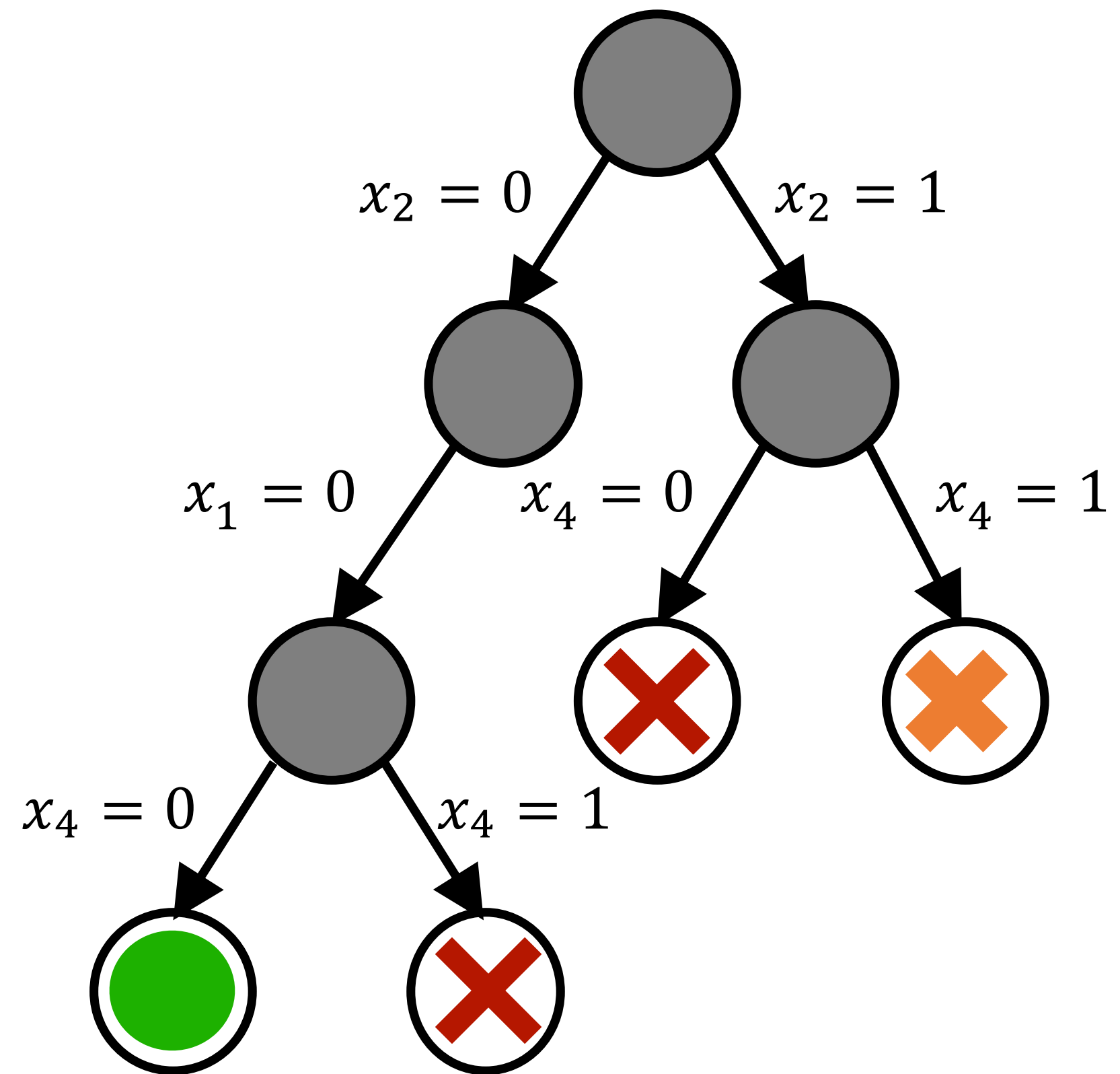
Jun 18, 2026

Backdoors

$$\min_x \mathbf{c}^T x \text{ s.t. } Ax \leq \mathbf{b}, x \in \{0,1\}^n, n \gg 3$$

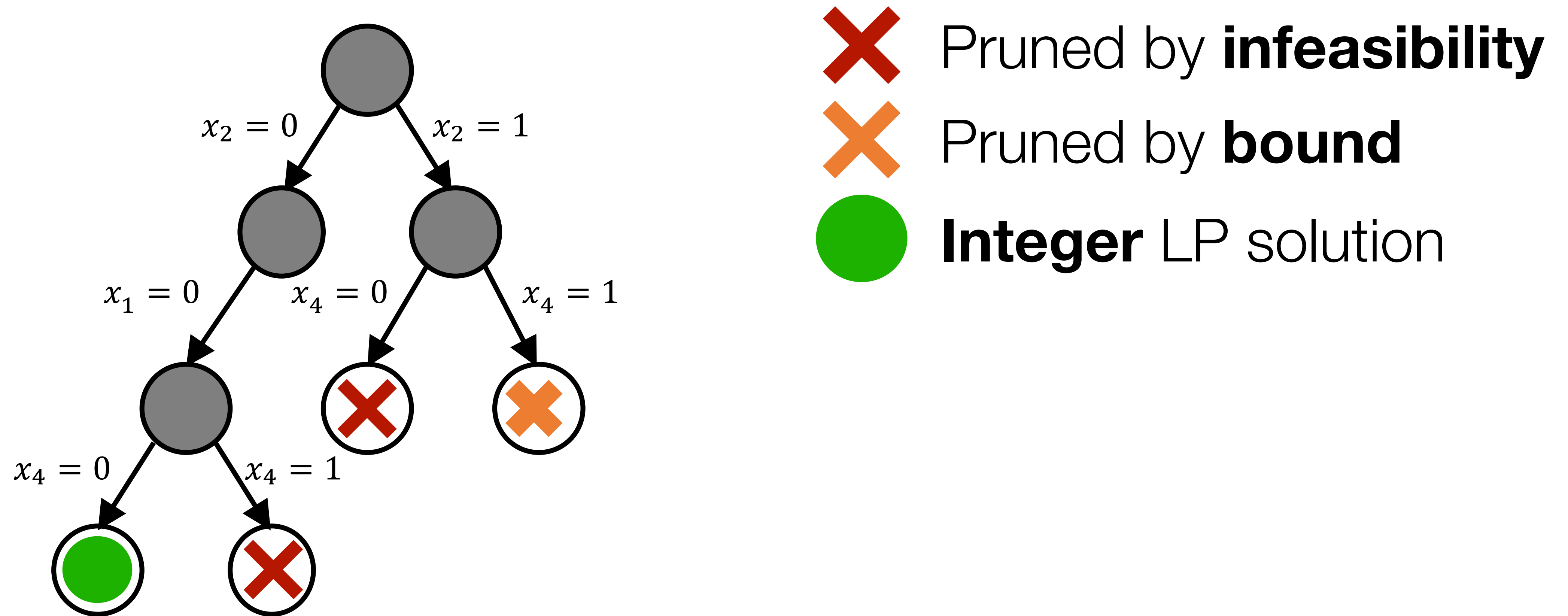
Backdoors

$$\min_x \mathbf{c}^T x \text{ s.t. } Ax \leq \mathbf{b}, x \in \{0,1\}^n, n \gg 3$$



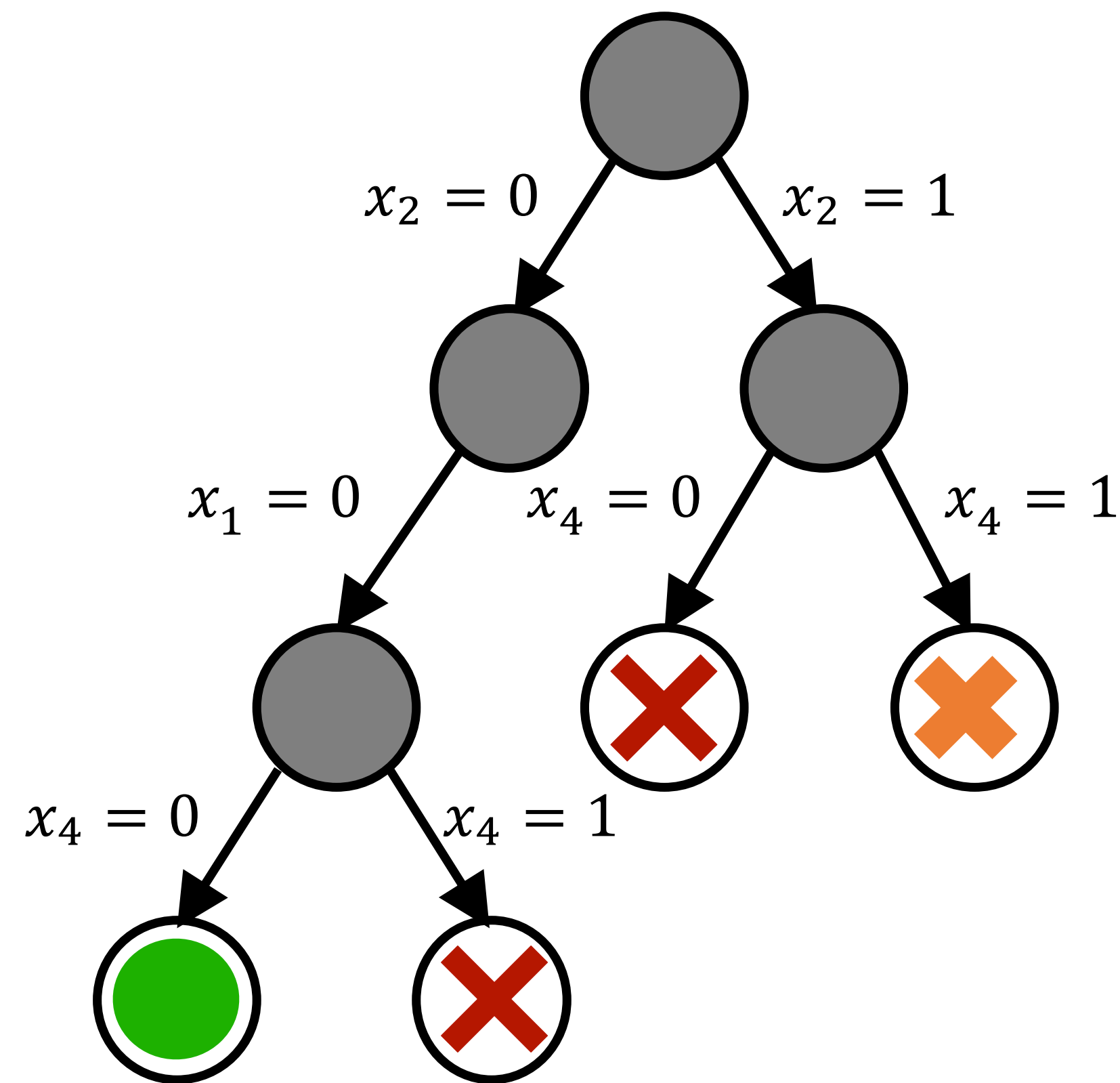
Backdoors

$$\min_x \mathbf{c}^T x \text{ s.t. } Ax \leq \mathbf{b}, x \in \{0,1\}^n, n \gg 3$$



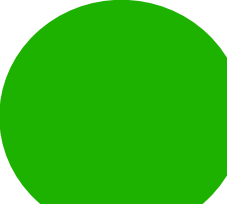
Backdoors

$$\min_x \mathbf{c}^T x \text{ s.t. } Ax \leq \mathbf{b}, x \in \{0,1\}^n, n \gg 3$$



 Pruned by **infeasibility**

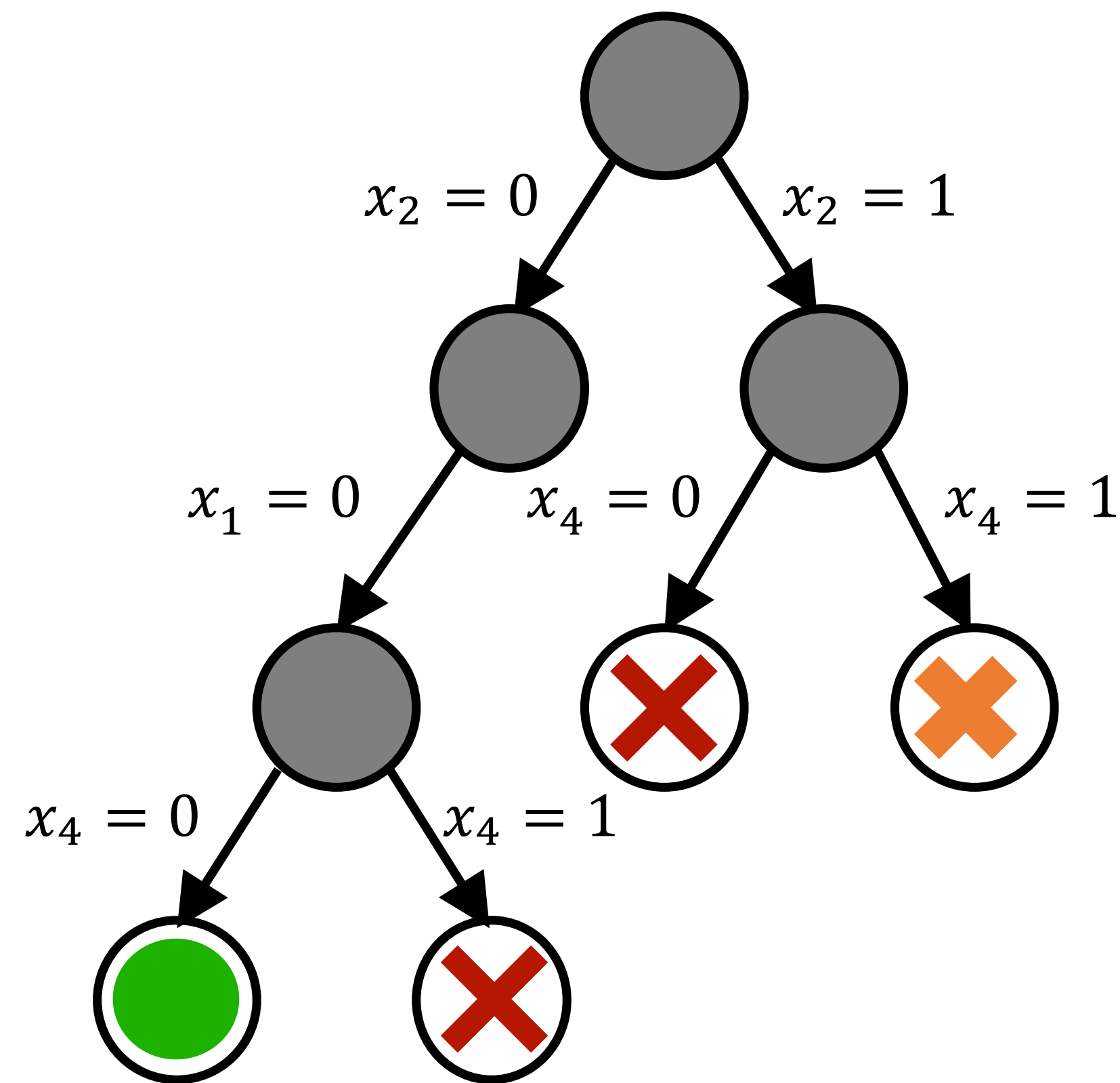
 Pruned by **bound**

 **Integer** LP solution

This is a “**certificate tree**”

Backdoors

$$\min_x \mathbf{c}^T x \text{ s.t. } Ax \leq \mathbf{b}, x \in \{0,1\}^n, n \gg 3$$

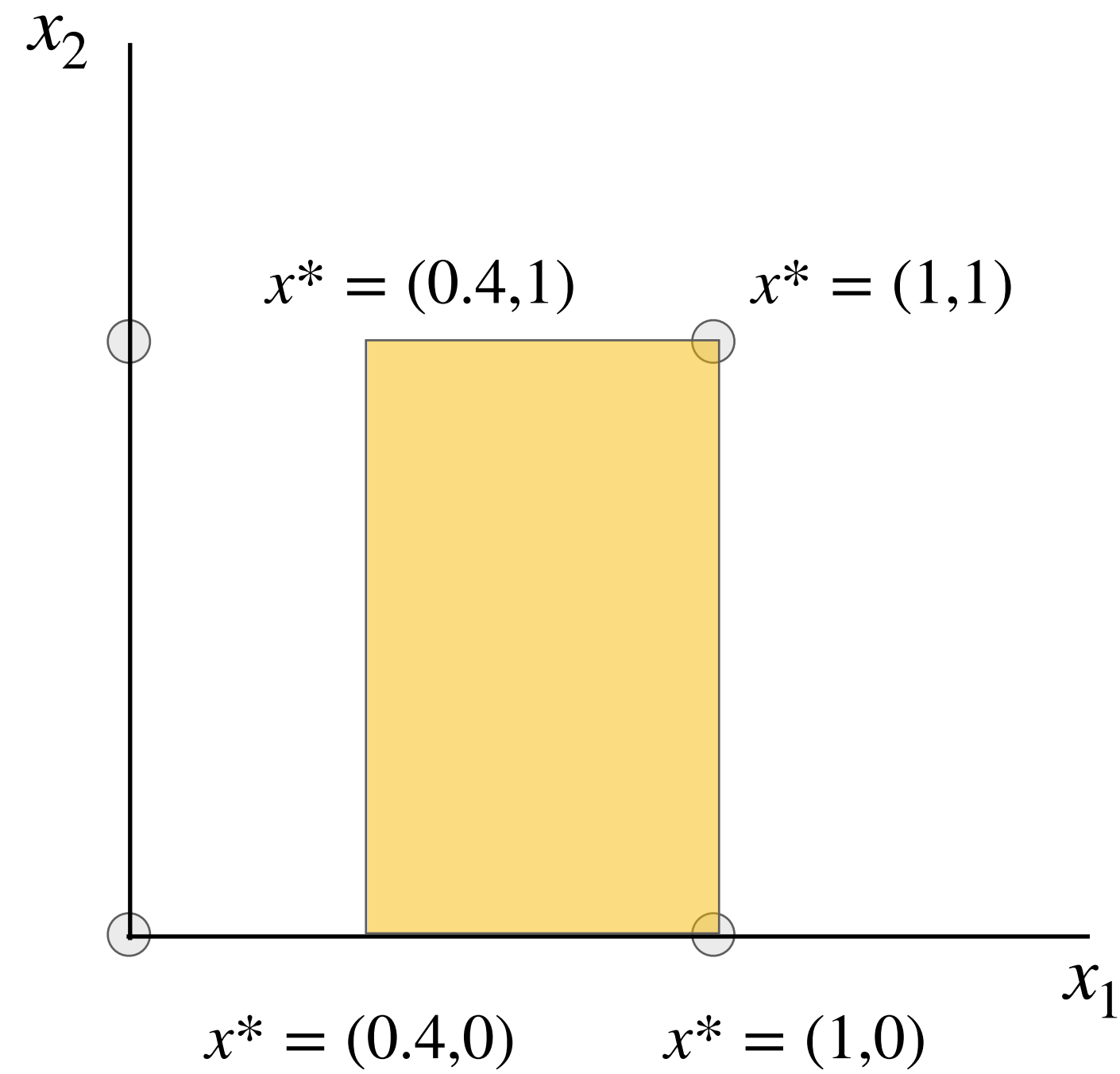


- ✗ Pruned by **infeasibility**
- ✗ Pruned by **bound**
- **Integer** LP solution

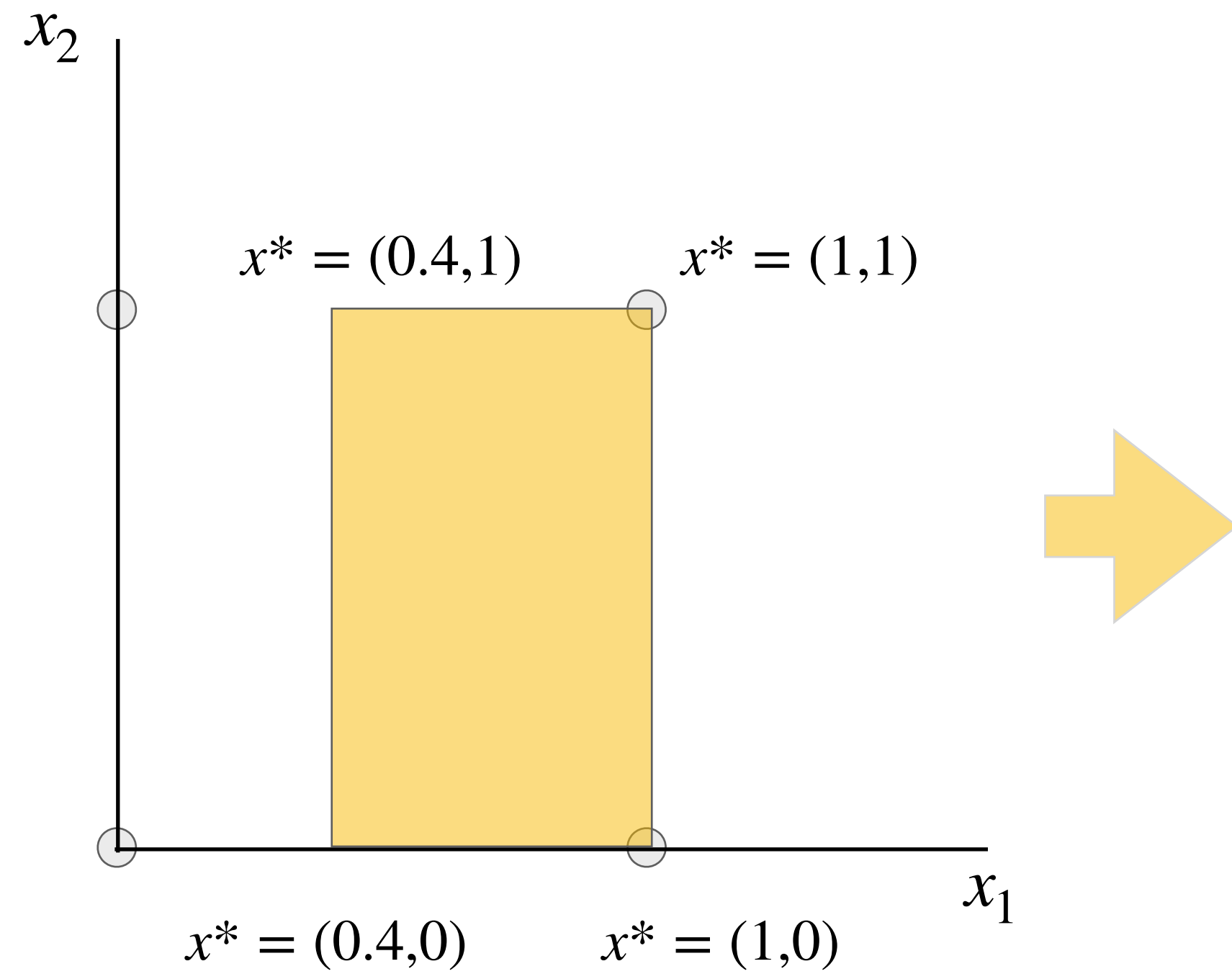
This is a “**certificate tree**”

$\{x_1, x_2, x_4\}$ is a **Backdoor!**

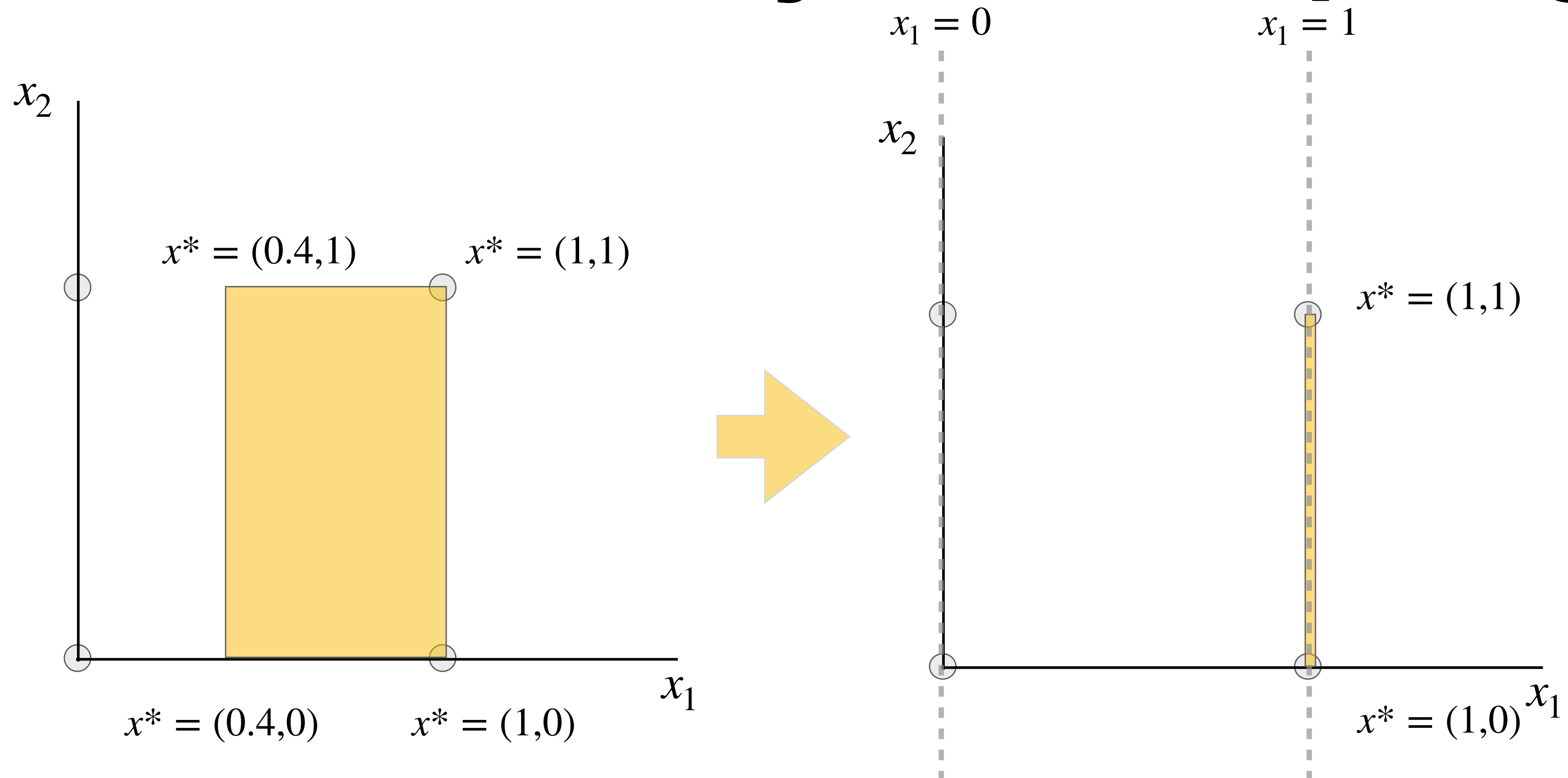
Binary linear program



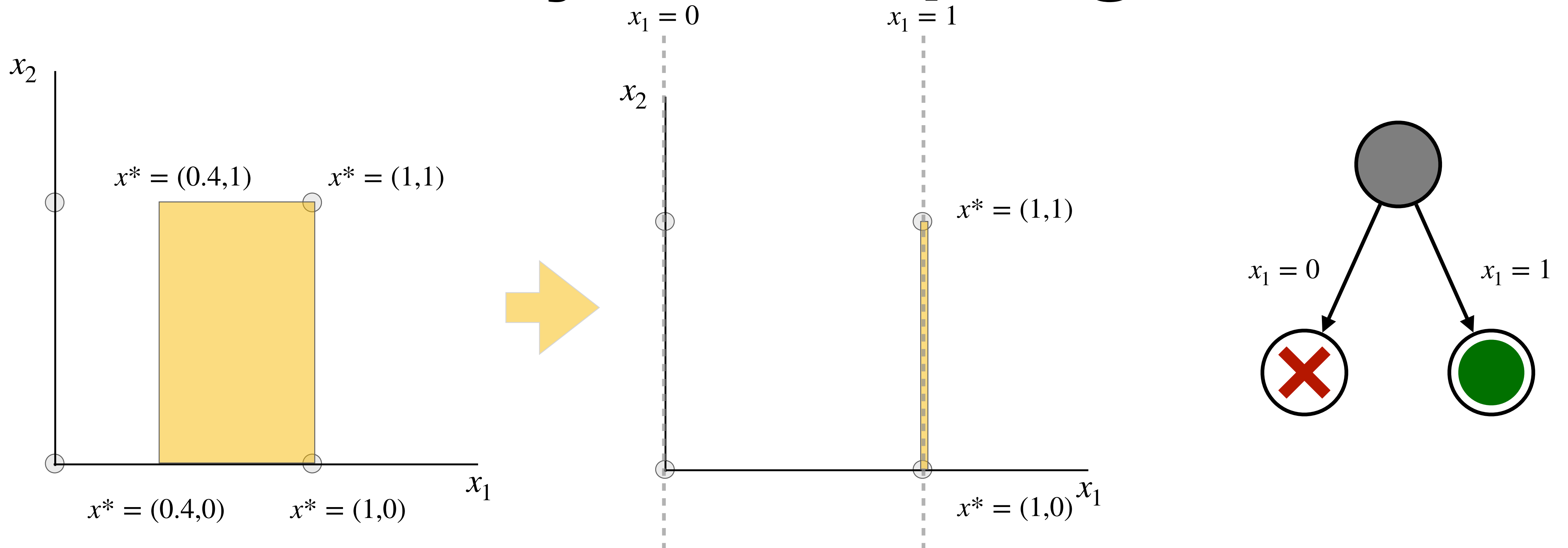
Binary linear program



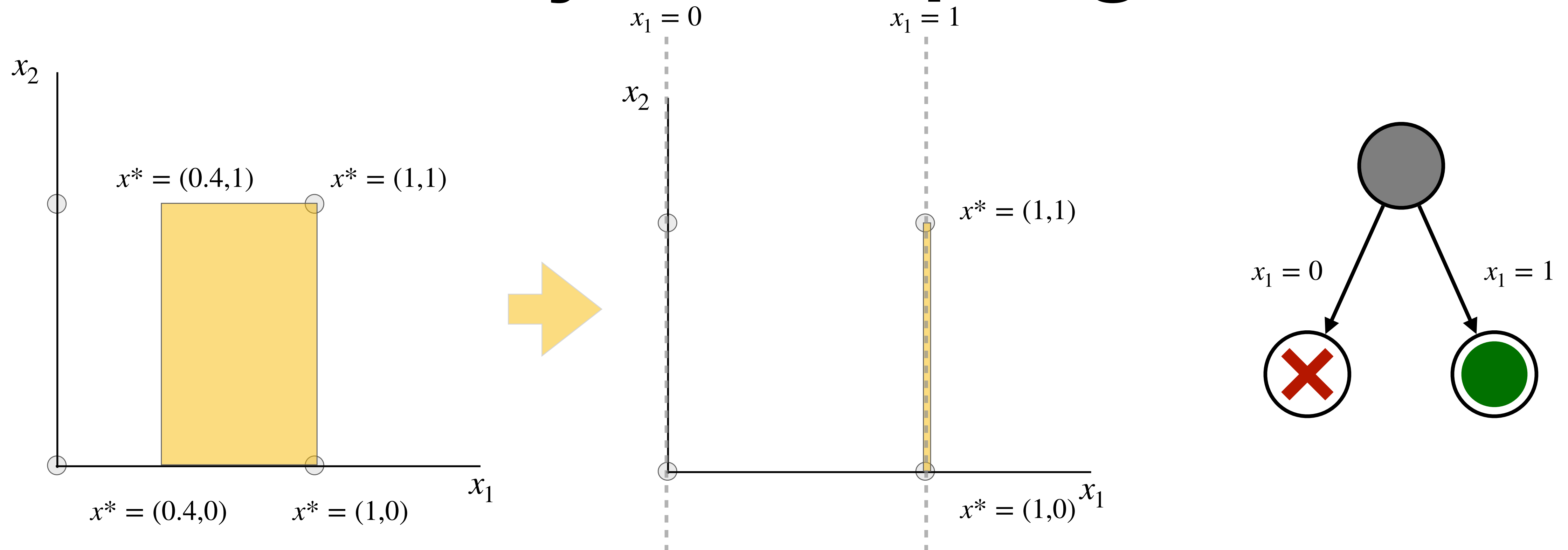
Binary linear program



Binary linear program



Binary linear program



Since we are working with 0-1 variables, no new vertices are introduced after branching. All integer feasible solutions are vertices of the polytope.

Hitting set model

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{array}{l} (\textcolor{red}{0.3}, \textcolor{red}{0.7}, 1, \textcolor{red}{0.2}, 0), \\ (0, 1, 1, 0, \textcolor{red}{0.5}), \\ (1, 0, 1, \textcolor{red}{0.6}, 0) \end{array} \right\}$$

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{array}{l} (\textcolor{red}{0.3}, \textcolor{red}{0.7}, 1, \textcolor{red}{0.2}, 0), \\ (0, 1, 1, 0, \textcolor{red}{0.5}), \\ (1, 0, 1, \textcolor{red}{0.6}, 0) \end{array} \right\}$$

$\mathcal{X} :=$ set of variables, $\{x_1, x_2, x_3, x_4, x_5\}$

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{array}{l} (\textcolor{red}{0.3}, \textcolor{red}{0.7}, 1, \textcolor{red}{0.2}, 0), \\ (0, 1, 1, 0, \textcolor{red}{0.5}), \\ (1, 0, 1, \textcolor{red}{0.6}, 0) \end{array} \right\}$$

$\mathcal{X} :=$ set of variables, $\{x_1, x_2, x_3, x_4, x_5\}$

$\mathcal{V}_{frac} :=$ set of fractional vertices, $\{v_1, v_2, v_3\}$

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{array}{l} (\textcolor{red}{0.3}, \textcolor{red}{0.7}, 1, \textcolor{red}{0.2}, 0), \\ (0, 1, 1, 0, \textcolor{red}{0.5}), \\ (1, 0, 1, \textcolor{red}{0.6}, 0) \end{array} \right\}$$

$\mathcal{X} :=$ set of variables, $\{x_1, x_2, x_3, x_4, x_5\}$

$\mathcal{V}_{frac} :=$ set of fractional vertices, $\{v_1, v_2, v_3\}$

$frac(v^*) := \{x_i \in \mathcal{X} : x_i \text{ is fractional in } v^*\}$

$frac(v_1) = \{x_1, x_2, x_4\}$

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{pmatrix} 0.3, & 0.7, & 1, & 0.2, & 0 \end{pmatrix}, \begin{pmatrix} 0, & 1, & 1, & 0, & 0.5 \end{pmatrix}, \begin{pmatrix} 1, & 0, & 1, & 0.6, & 0 \end{pmatrix} \right\}$$

Binary variable y_i such that $y_i = 1$ if variable $x_i \in \mathcal{X}$ is in a backdoor set and $= 0$ otherwise

$\mathcal{X} :=$ set of variables, $\{x_1, x_2, x_3, x_4, x_5\}$

$\mathcal{V}_{frac} :=$ set of fractional vertices, $\{v_1, v_2, v_3\}$

$frac(v^*) := \{x_i \in \mathcal{X} : x_i \text{ is fractional in } v^*\}$

$frac(v_1) = \{x_1, x_2, x_4\}$

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{pmatrix} 0.3, & 0.7, & 1, & 0.2, & 0 \end{pmatrix}, \begin{pmatrix} 0, & 1, & 1, & 0, & 0.5 \end{pmatrix}, \begin{pmatrix} 1, & 0, & 1, & 0.6, & 0 \end{pmatrix} \right\}$$

Binary variable y_i such that $y_i = 1$ if variable $x_i \in \mathcal{X}$ is in a backdoor set and $= 0$ otherwise

$$\min \sum_{i \in \mathcal{X}} y_i$$

$$\text{s.t.} \quad \sum_{i \in frac(v^*)} y_i \geq 1, \quad \forall v^* \in \mathcal{V}_{frac}$$

$$y_i \in \{0, 1\}, \quad \forall i \in \mathcal{X}$$

$\mathcal{X} :=$ set of variables, $\{x_1, x_2, x_3, x_4, x_5\}$

$\mathcal{V}_{frac} :=$ set of fractional vertices, $\{v_1, v_2, v_3\}$

$frac(v^*) := \{x_i \in \mathcal{X} : x_i \text{ is fractional in } v^*\}$

$frac(v_1) = \{x_1, x_2, x_4\}$

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{pmatrix} 0.3, & 0.7, & 1, & 0.2, & 0 \end{pmatrix}, \begin{pmatrix} 0, & 1, & 1, & 0, & 0.5 \end{pmatrix}, \begin{pmatrix} 1, & 0, & 1, & 0.6, & 0 \end{pmatrix} \right\}$$

Binary variable y_i such that $y_i = 1$ if variable $x_i \in \mathcal{X}$ is in a backdoor set and $= 0$ otherwise

$$\min \sum_{i \in \mathcal{X}} y_i$$

Solving this problem finds the smallest strong backdoor

$$\text{s.t.} \quad \sum_{i \in \text{frac}(v^*)} y_i \geq 1, \quad \forall v^* \in \mathcal{V}_{frac}$$

$$y_i \in \{0, 1\}, \quad \forall i \in \mathcal{X}$$

$\mathcal{X} :=$ set of variables, $\{x_1, x_2, x_3, x_4, x_5\}$

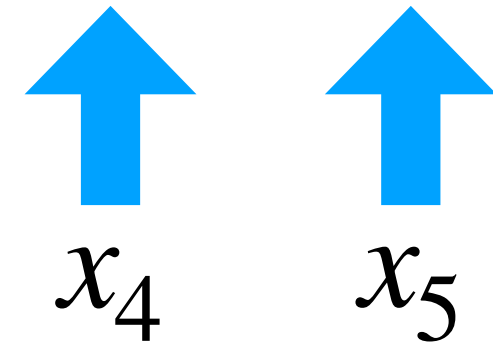
$\mathcal{V}_{frac} :=$ set of fractional vertices, $\{v_1, v_2, v_3\}$

$\text{frac}(v^*) := \{x_i \in \mathcal{X} : x_i \text{ is fractional in } v^*\}$

$\text{frac}(v_1) = \{x_1, x_2, x_4\}$

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{pmatrix} 0.3 & 0.7 & 1 & 0.2 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 1 & 1 & 0 & 0.5 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 1 & 0.6 & 0 \end{pmatrix} \right\}$$



$\mathcal{X} :=$ set of variables, $\{x_1, x_2, x_3, x_4, x_5\}$

$\mathcal{V}_{frac} :=$ set of fractional vertices, $\{v_1, v_2, v_3\}$

$frac(v^*) := \{x_i \in \mathcal{X} : x_i \text{ is fractional in } v^*\}$

$frac(v_1) = \{x_1, x_2, x_4\}$

Binary variable y_i such that $y_i = 1$ if variable $x_i \in \mathcal{X}$ is in a backdoor set and $= 0$ otherwise

$$\min \sum_{i \in \mathcal{X}} y_i$$

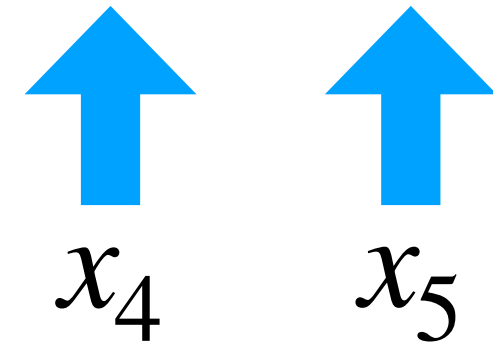
Solving this problem finds the smallest strong backdoor

$$\text{s.t. } \sum_{i \in frac(v^*)} y_i \geq 1, \quad \forall v^* \in \mathcal{V}_{frac}$$

$$y_i \in \{0, 1\}, \quad \forall i \in \mathcal{X}$$

Hitting set model

$$\mathcal{V}_{frac} = \left\{ \begin{pmatrix} 0.3 & 0.7 & 1 & 0.2 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 1 & 1 & 0 & 0.5 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 1 & 0.6 & 0 \end{pmatrix} \right\}$$



$\mathcal{X} :=$ set of variables, $\{x_1, x_2, x_3, x_4, x_5\}$

$\mathcal{V}_{frac} :=$ set of fractional vertices, $\{v_1, v_2, v_3\}$

$frac(v^*) := \{x_i \in \mathcal{X} : x_i \text{ is fractional in } v^*\}$

$frac(v_1) = \{x_1, x_2, x_4\}$

Binary variable y_i such that $y_i = 1$ if variable $x_i \in \mathcal{X}$ is in a backdoor set and $= 0$ otherwise

$$\min \sum_{i \in \mathcal{X}} y_i$$

Solving this problem finds the smallest strong backdoor

$$\text{s.t. } \sum_{i \in frac(v^*)} y_i \geq 1, \quad \forall v^* \in \mathcal{V}_{frac}$$

$$y_i \in \{0, 1\}, \quad \forall i \in \mathcal{X}$$

$$y^* = (0, 0, 0, 1, 1)$$

$$\text{Backdoor} = \{x_4, x_5\}$$

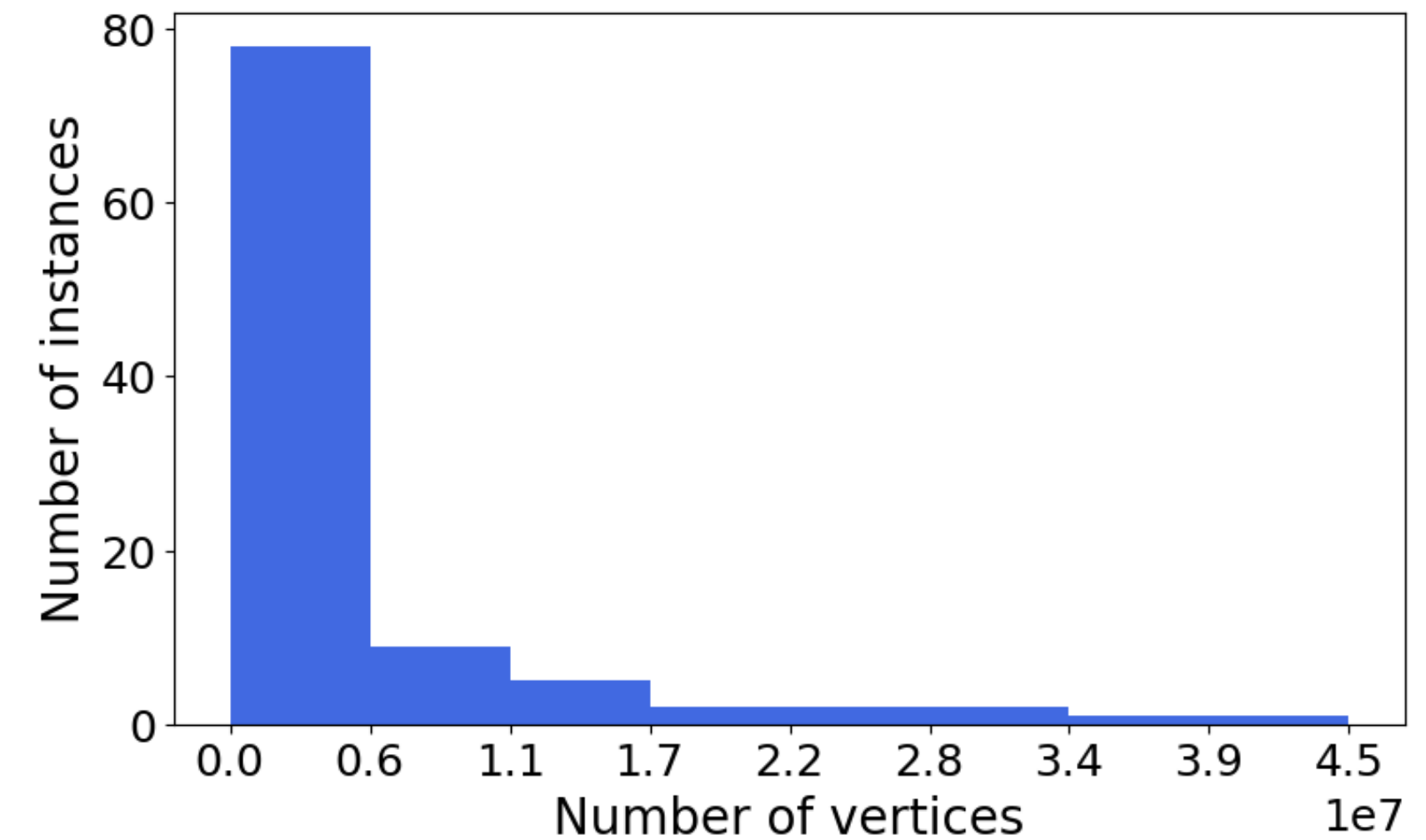
Backdoors on small instances

Backdoors on small instances

1. Generated 100 integer-infeasible set covering instances, with 20 variables and 40 constraints.

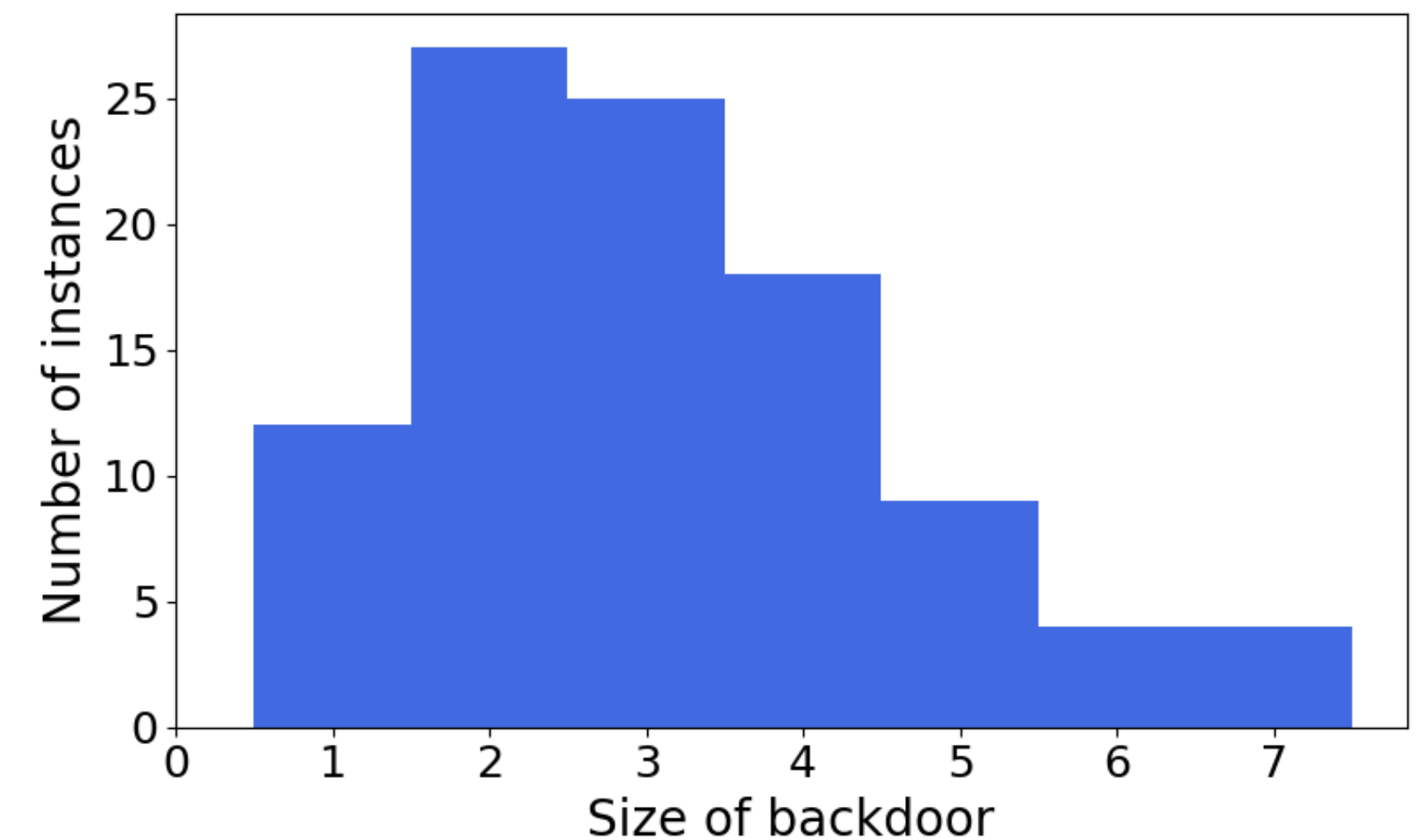
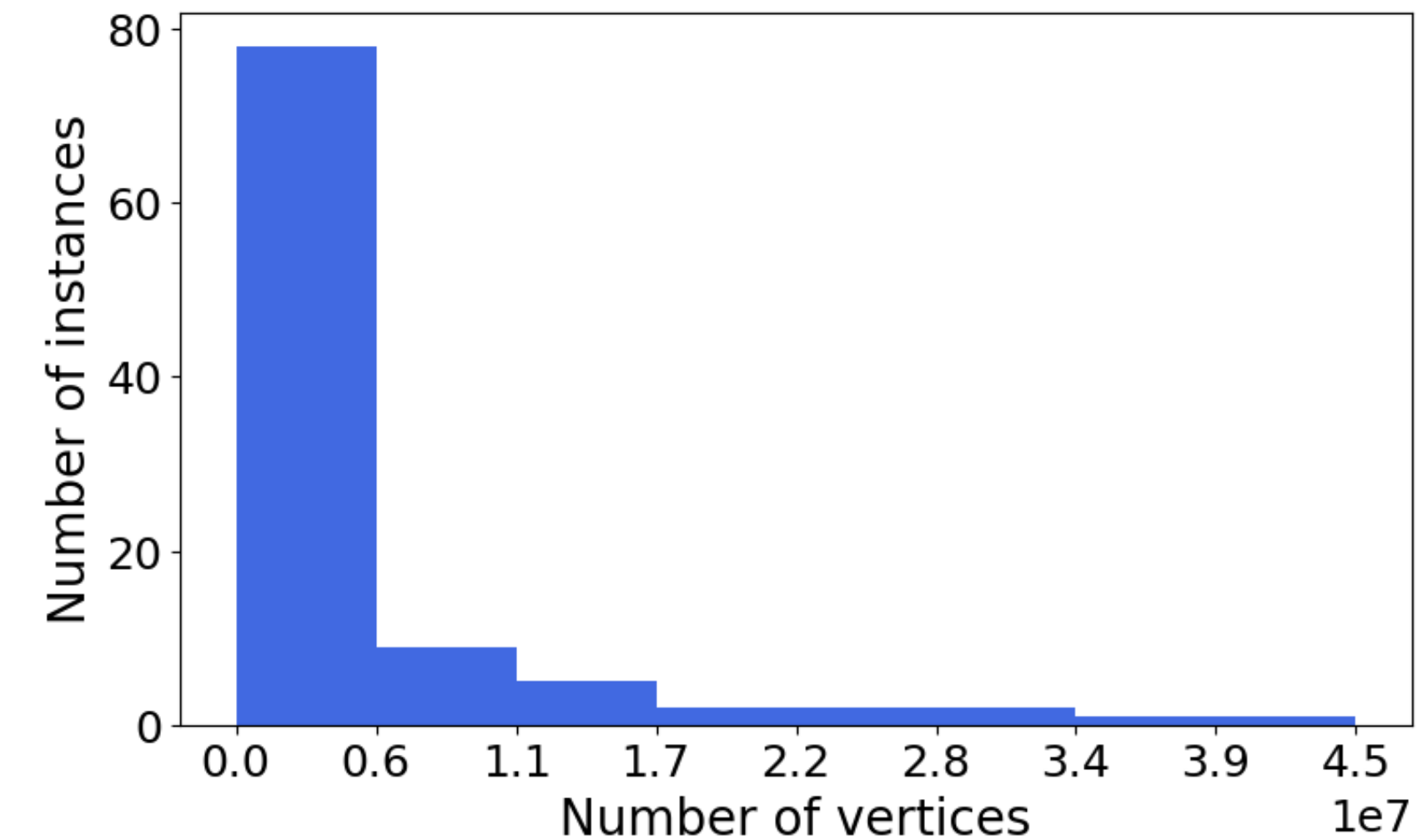
Backdoors on small instances

1. Generated 100 integer-infeasible set covering instances, with 20 variables and 40 constraints.
2. On average, each instance had 2M vertices with min of 726 to max of 8M



Backdoors on small instances

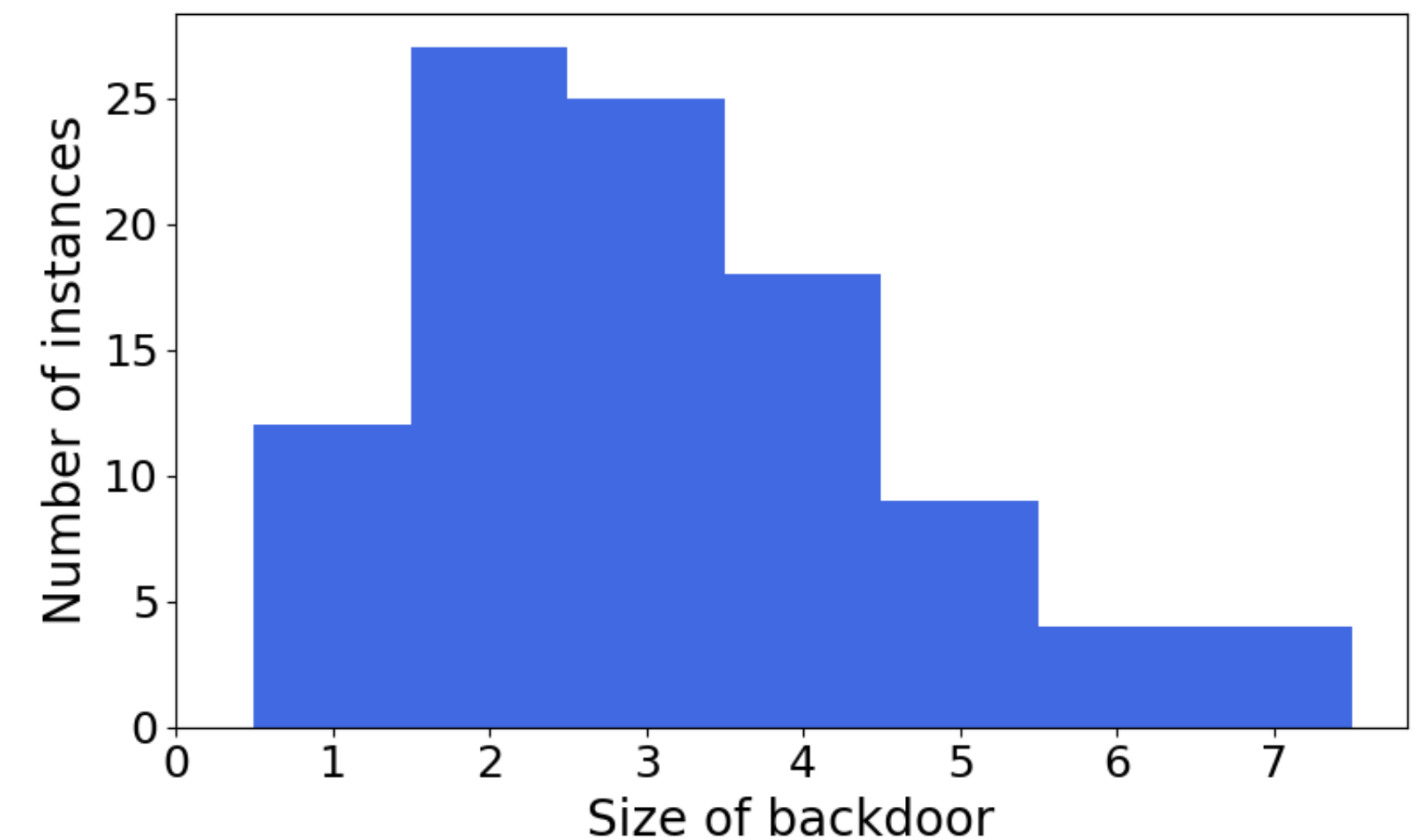
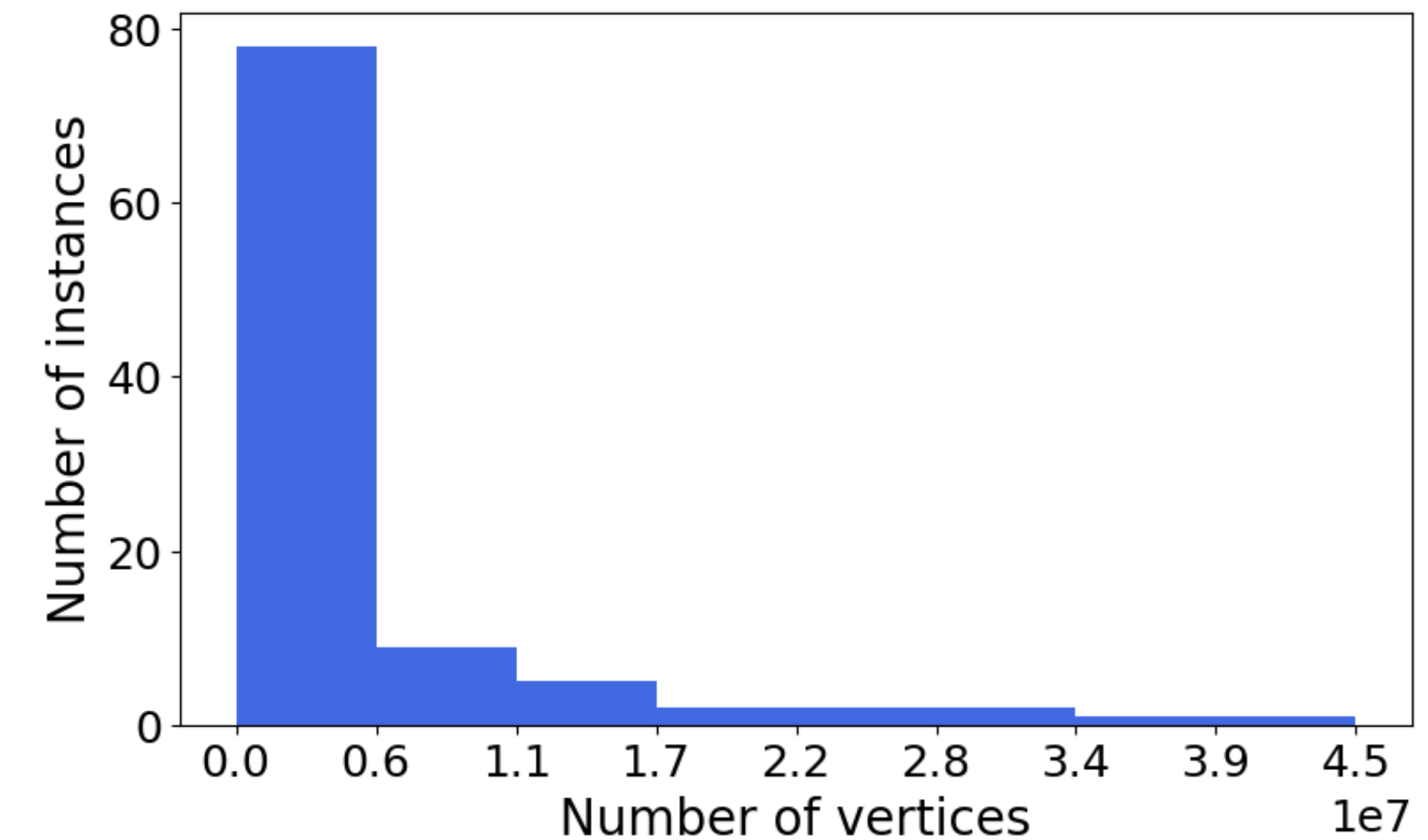
1. Generated 100 integer-infeasible set covering instances, with 20 variables and 40 constraints.
2. On average, each instance had 2M vertices with min of 726 to max of 8M
3. Median backdoor size is 3 with min 1 and max of 7



Backdoors on small instances

1. Generated 100 integer-infeasible set covering instances, with 20 variables and 40 constraints.
2. On average, each instance had 2M vertices with min of 726 to max of 8M
3. Median backdoor size is 3 with min 1 and max of 7

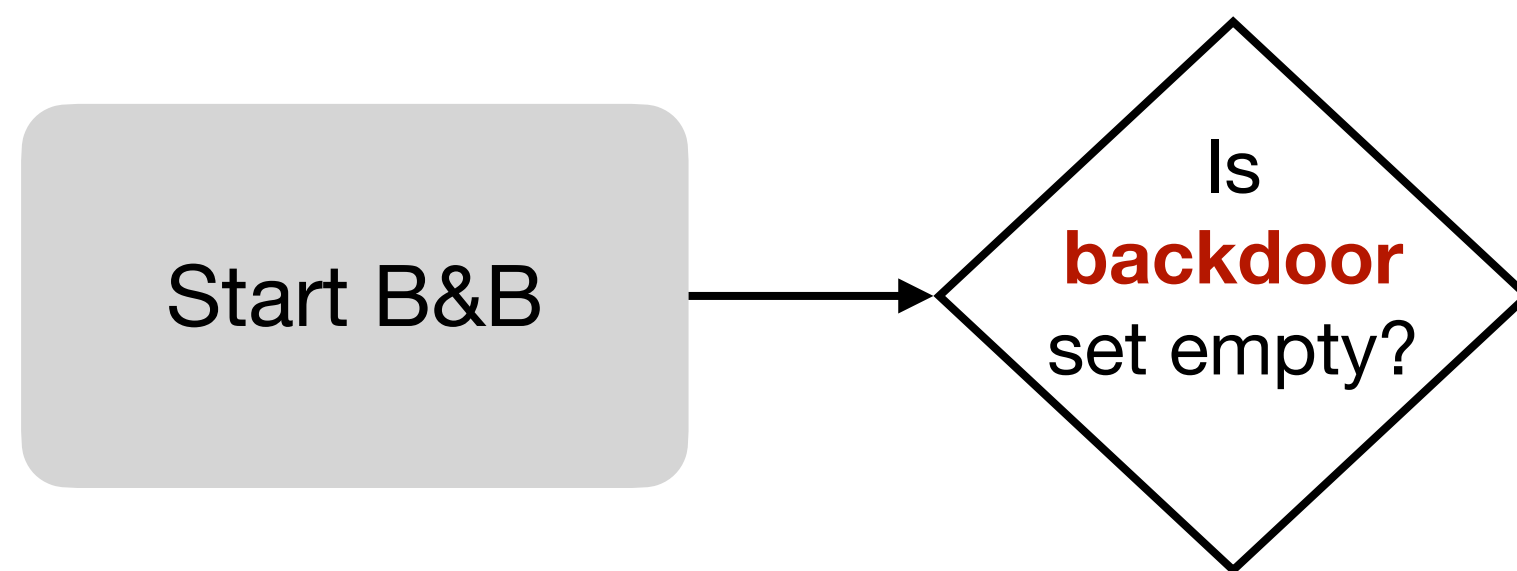
For these small instances, small backdoor do exist.



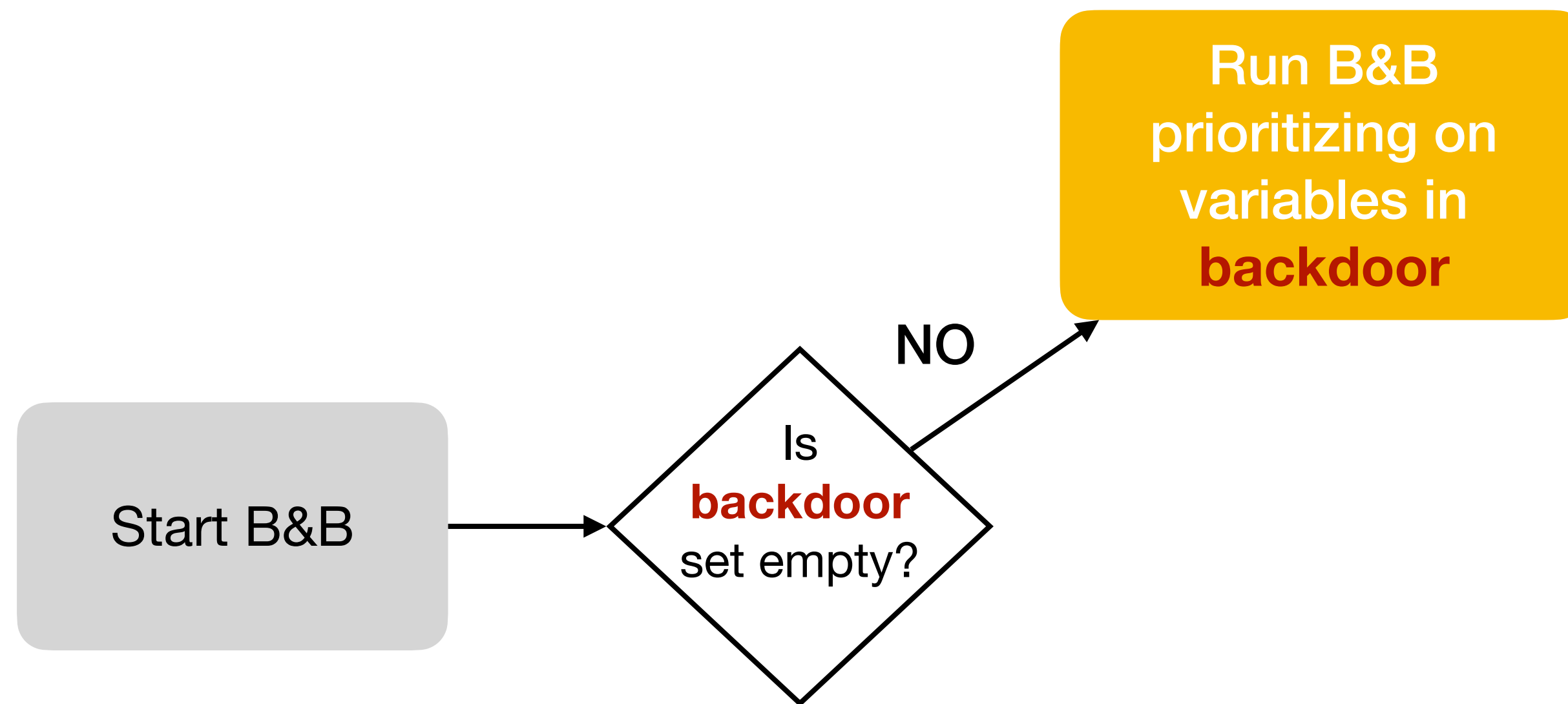
Set covering algorithm overview

Start B&B

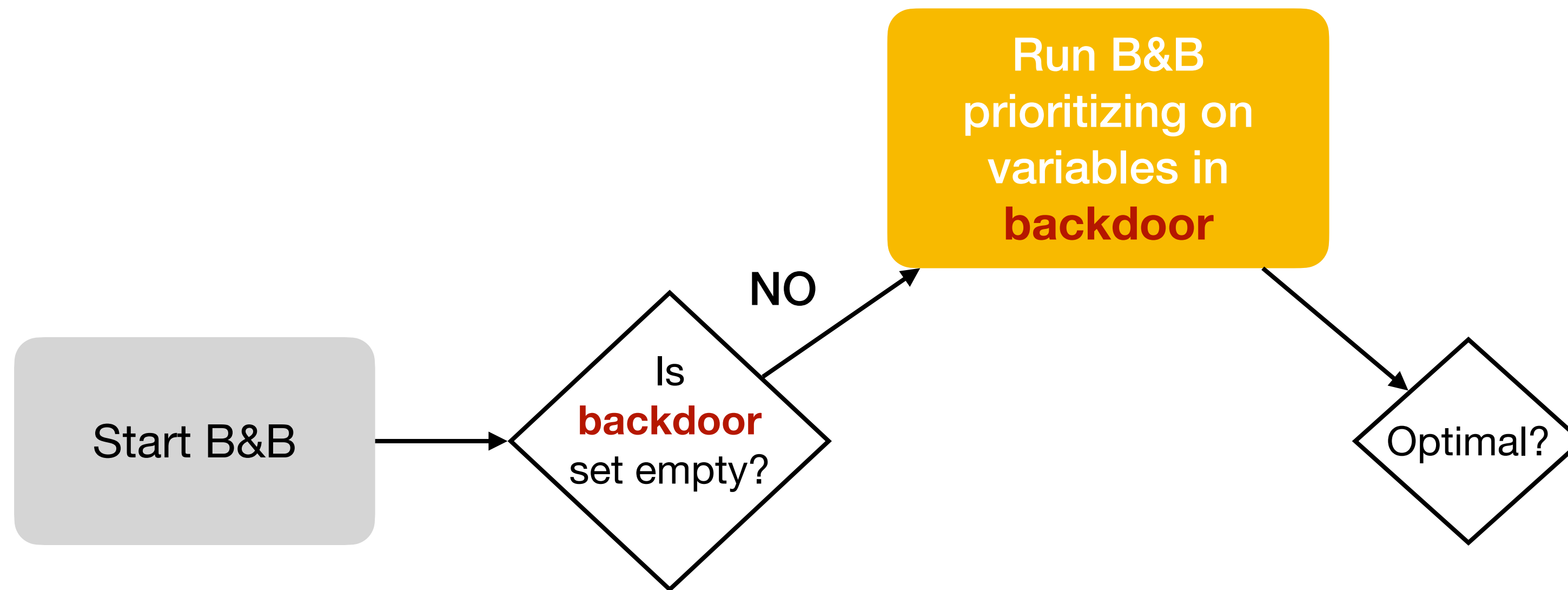
Set covering algorithm overview



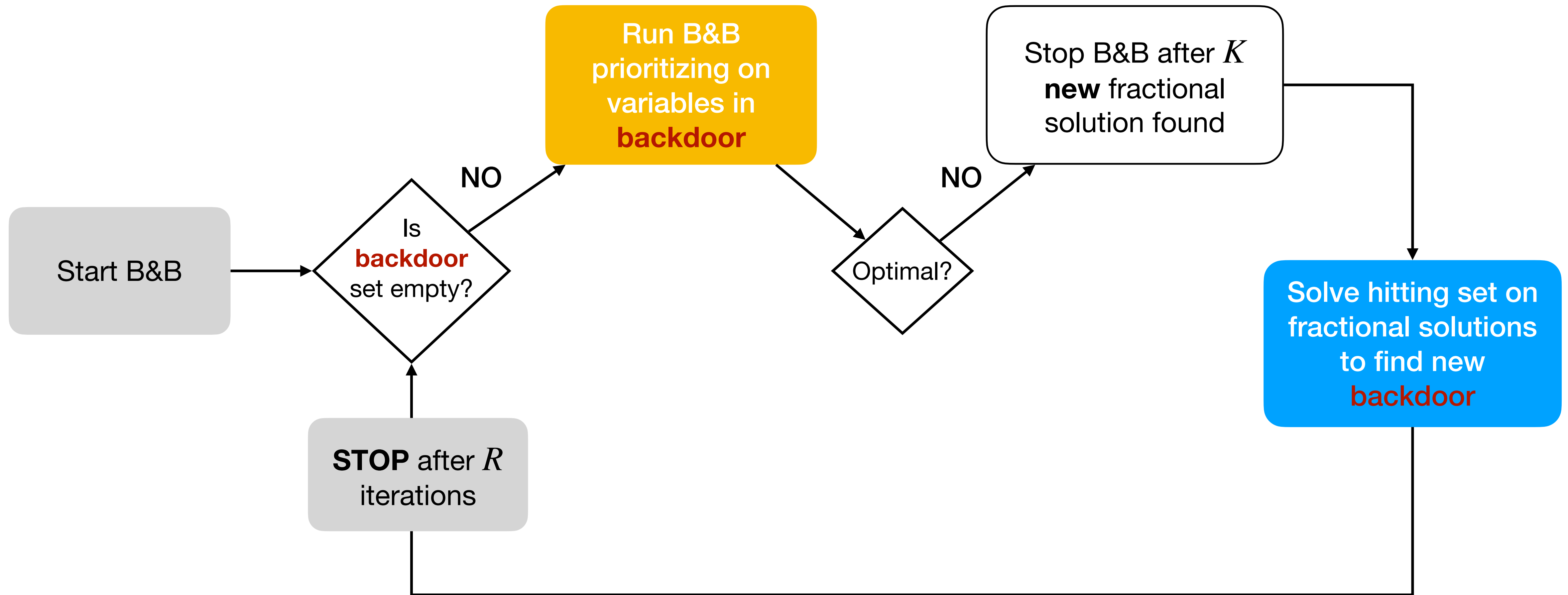
Set covering algorithm overview



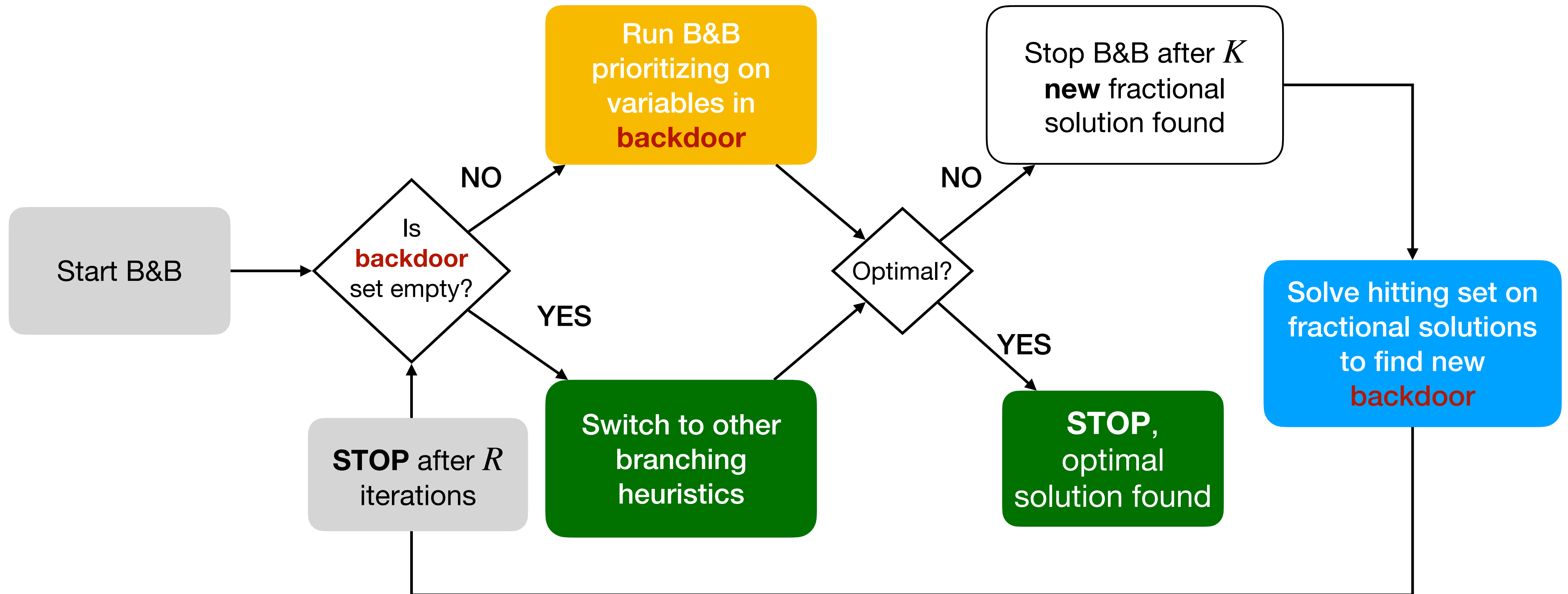
Set covering algorithm overview



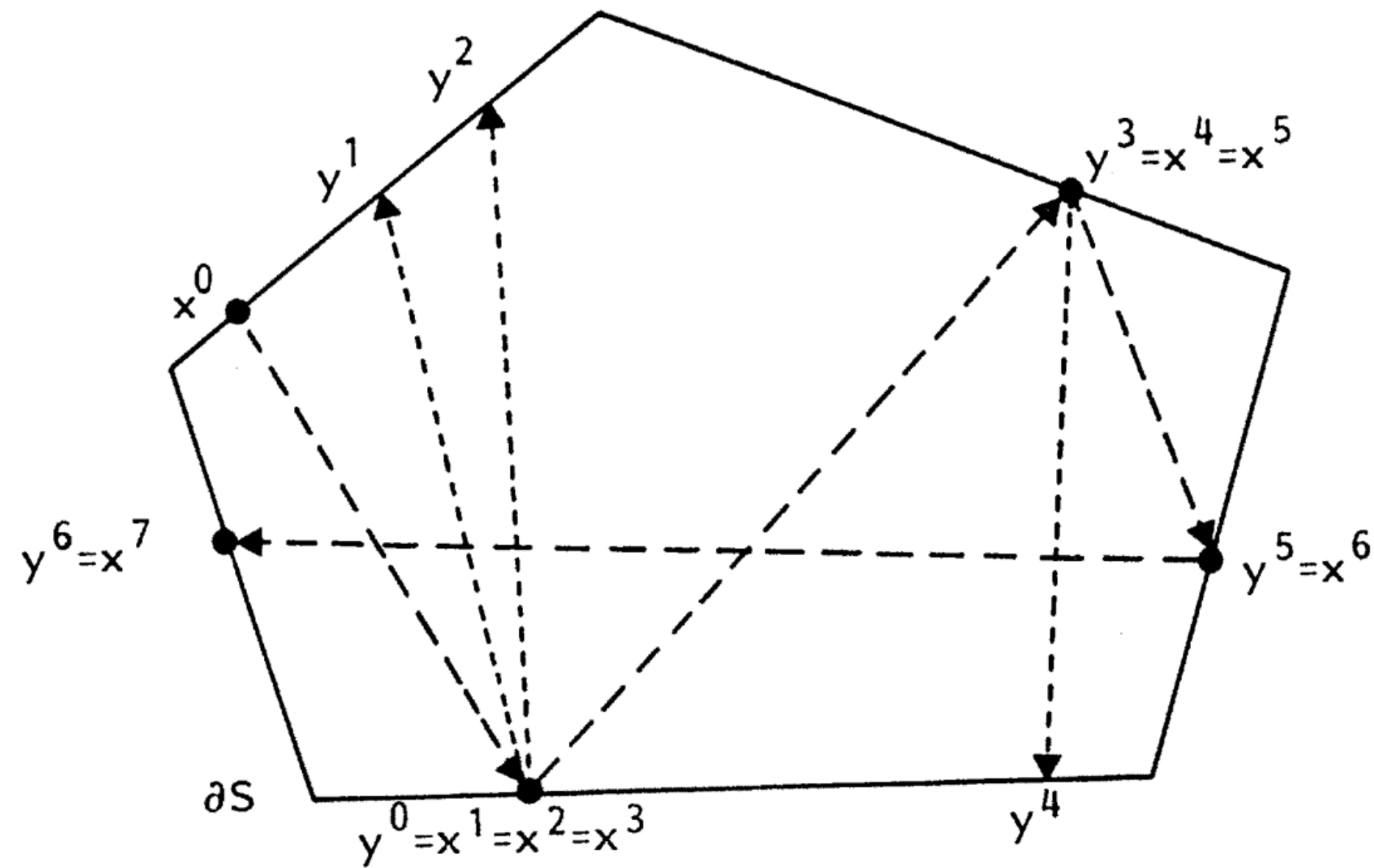
Set covering algorithm overview



Set covering algorithm overview



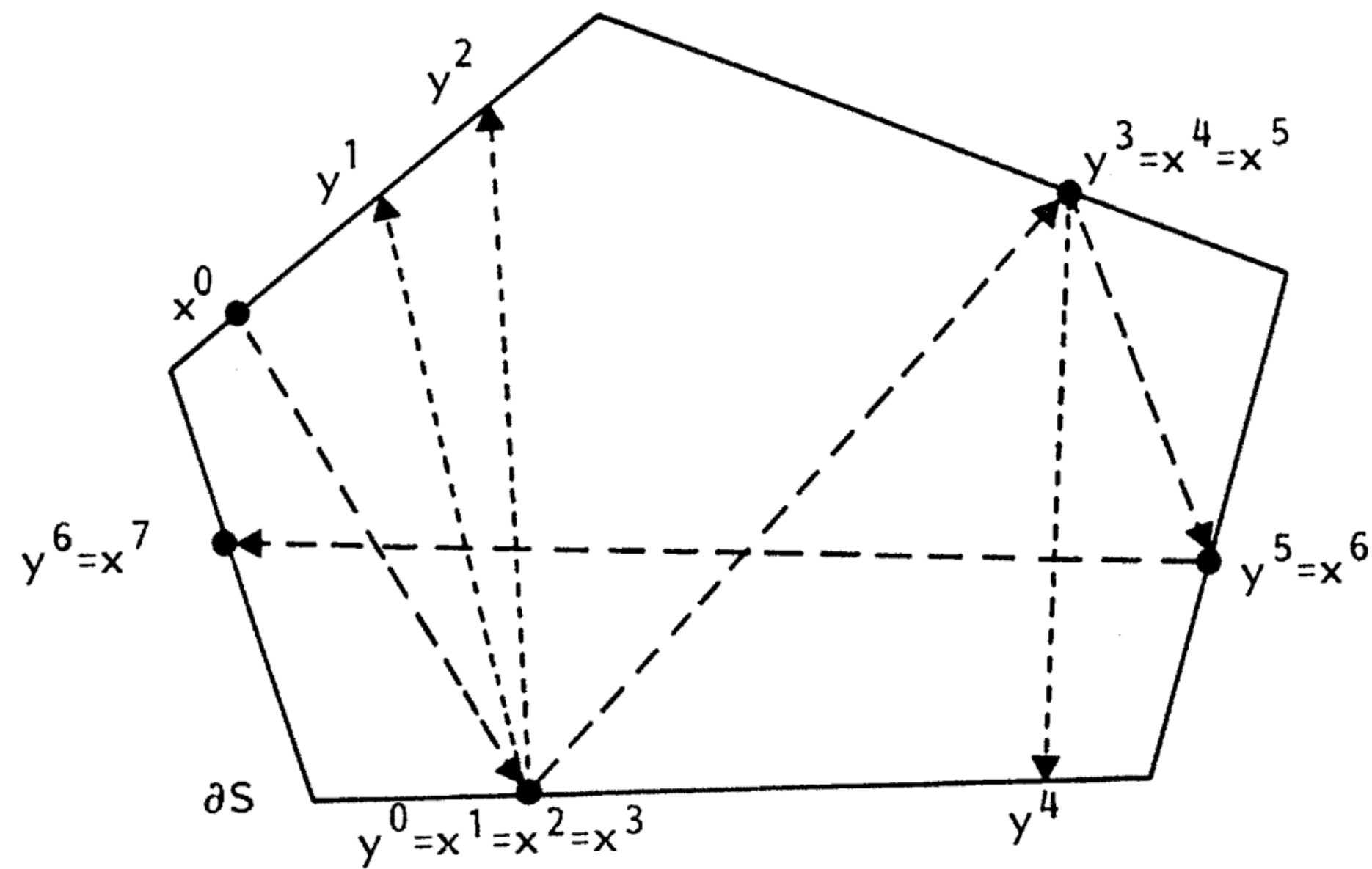
Shake-and-bake (SB)



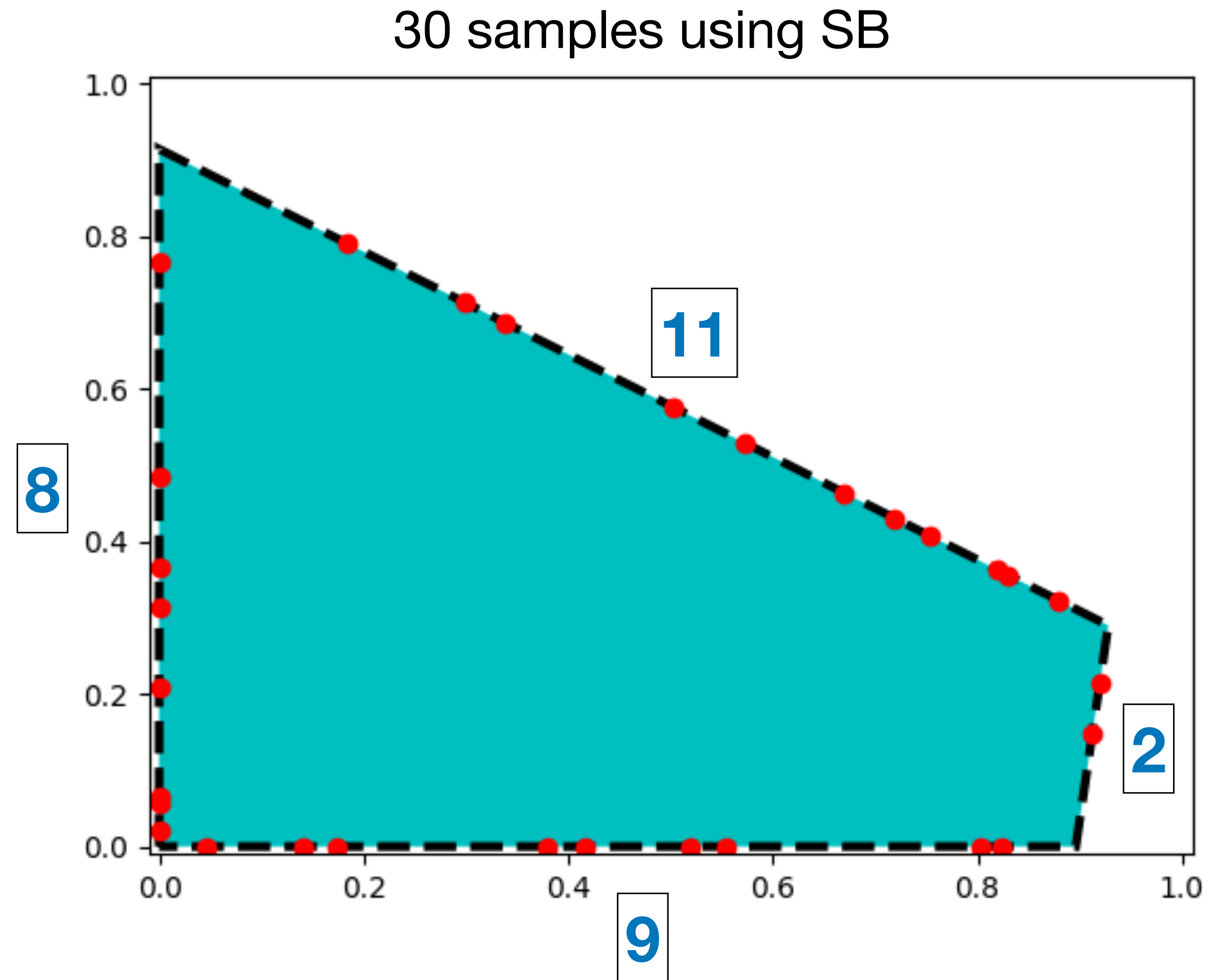
- Start at x^0
- Iter 0: y^0 is accepted
- Iter 1: y^1 is rejected
- Iter 2: y^2 is rejected
- Iter 3: y^3 is accepted

Boender et al. "Shake-and-bake algorithms for generating uniform points on the boundary of bounded polyhedra". Operations research, 1991.

Shake-and-bake (SB)

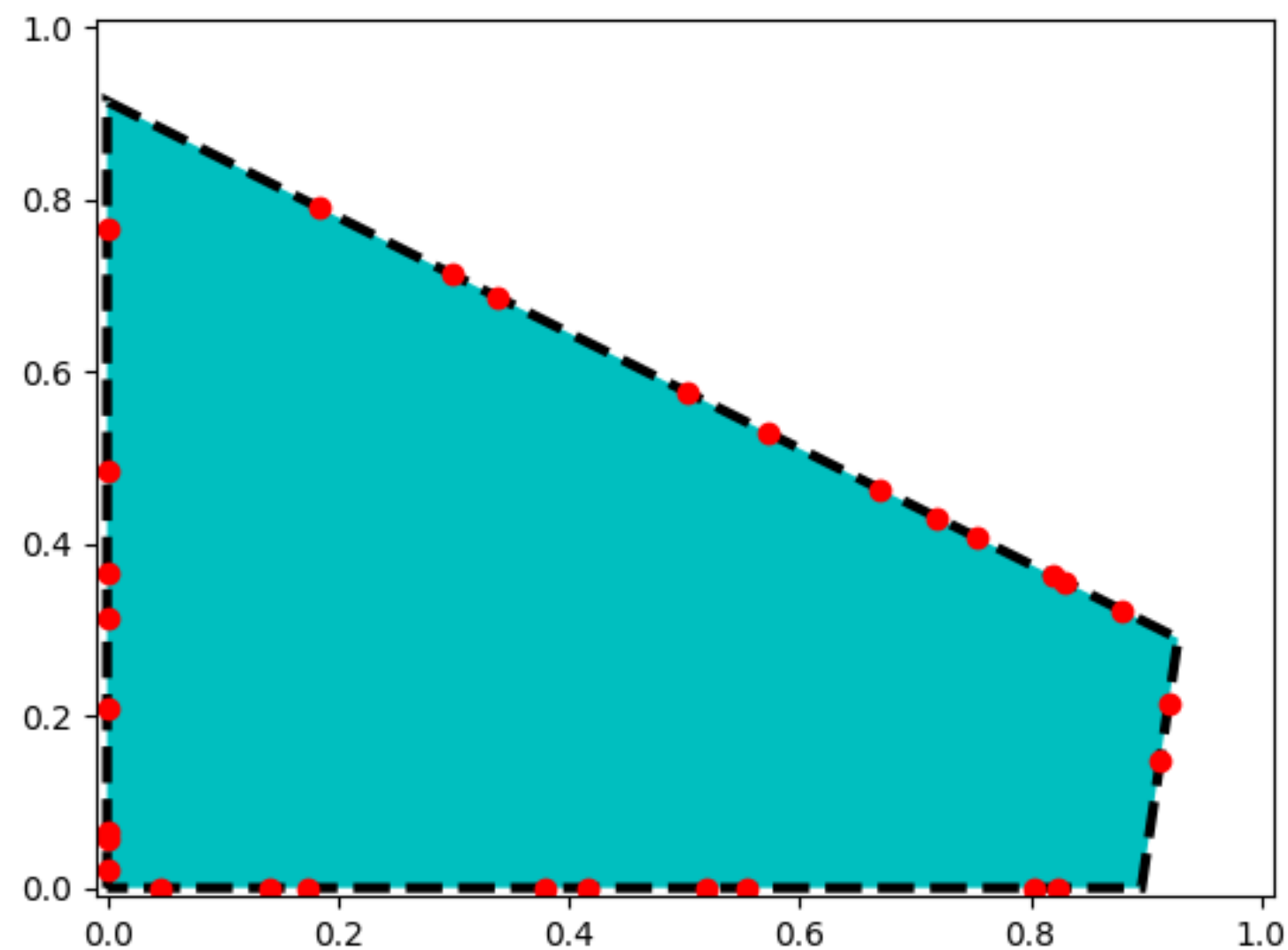


- Start at x^0
- Iter 0: y^0 is accepted
- Iter 1: y^1 is rejected
- Iter 2: y^2 is rejected
- Iter 3: y^3 is accepted

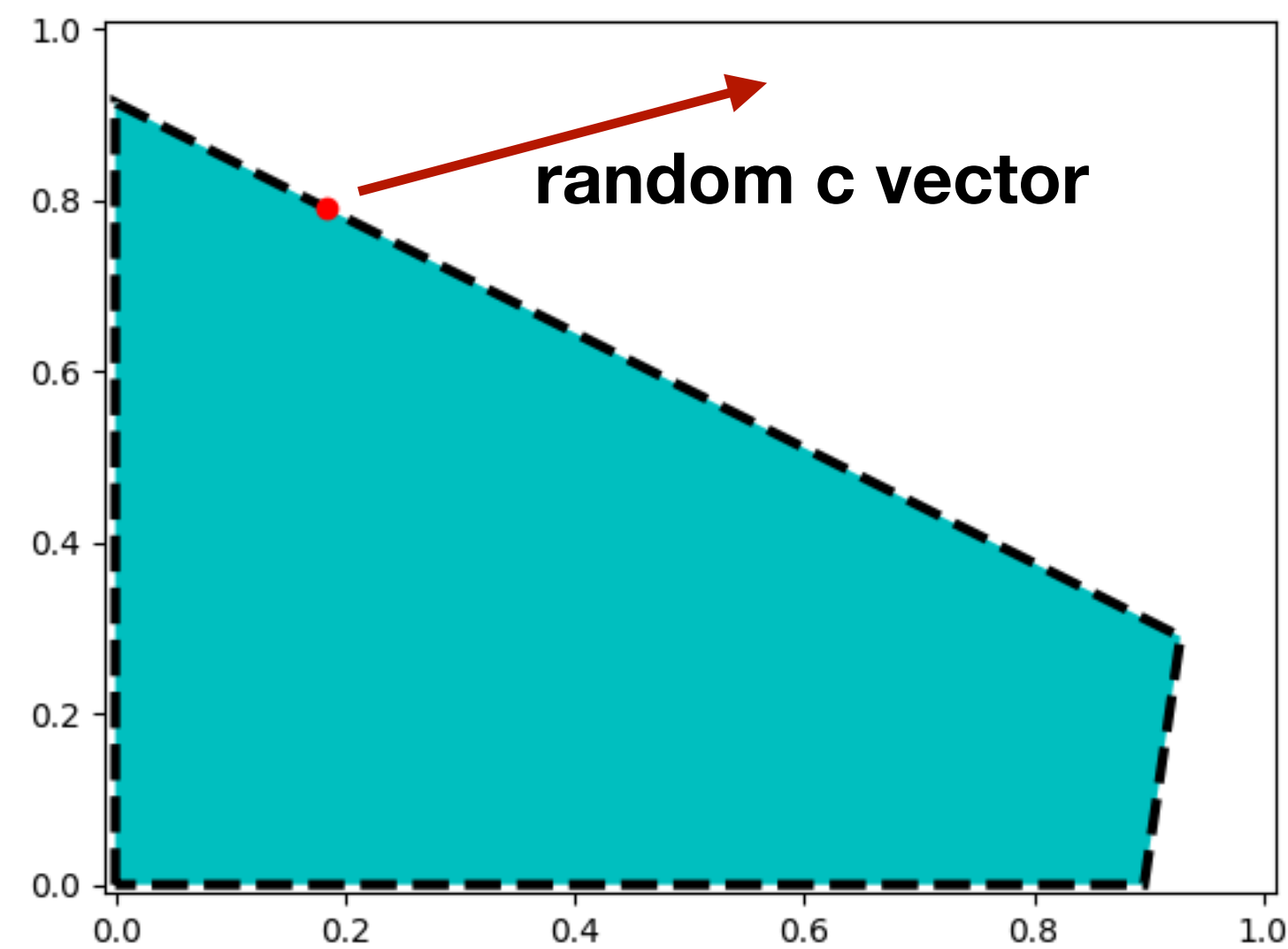


Boender et al. "Shake-and-bake algorithms for generating uniform points on the boundary of bounded polyhedra". Operations research, 1991.

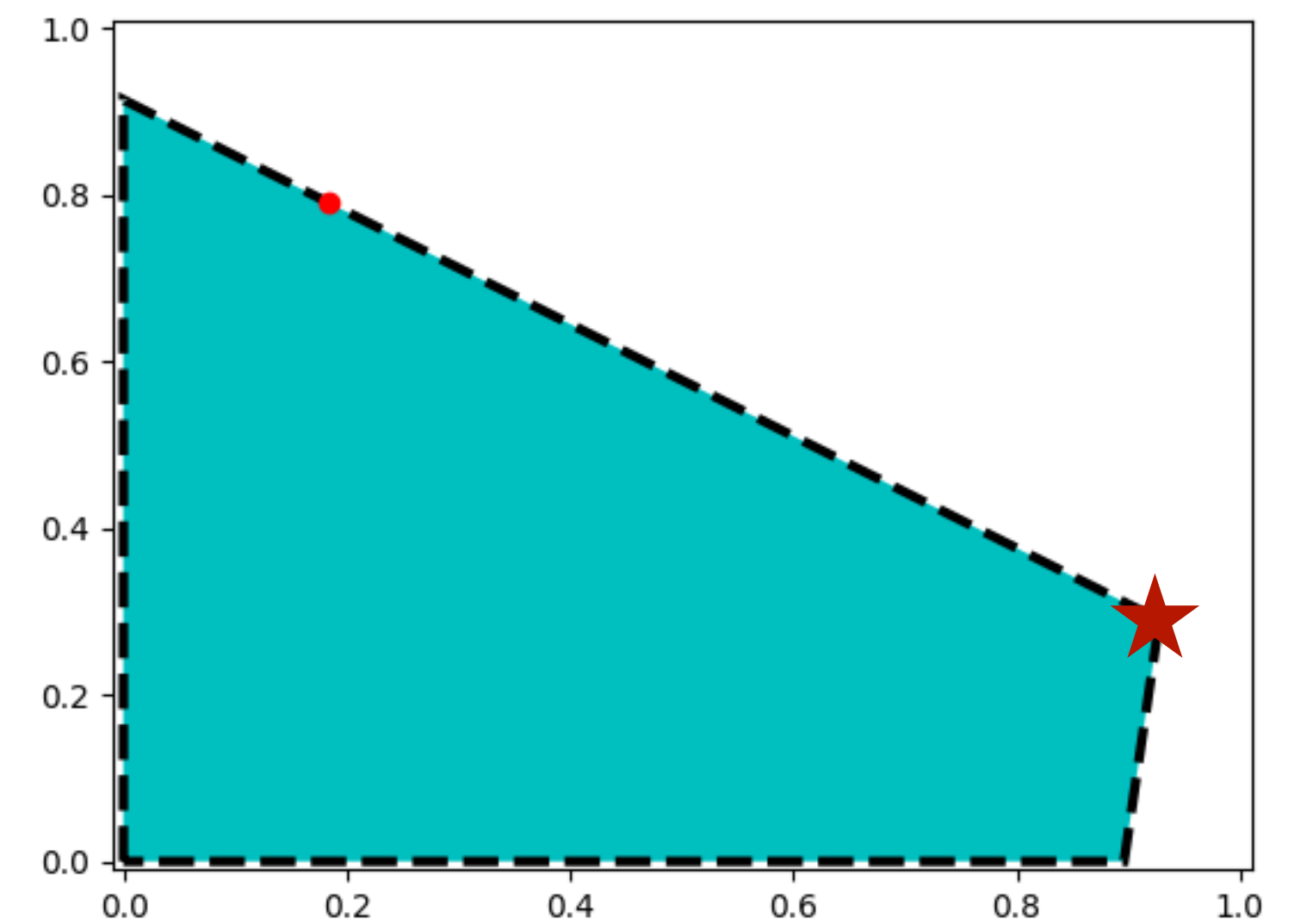
Modified shake-and-bake (MSB)



Step 1: generate samples on the boundaries using SB

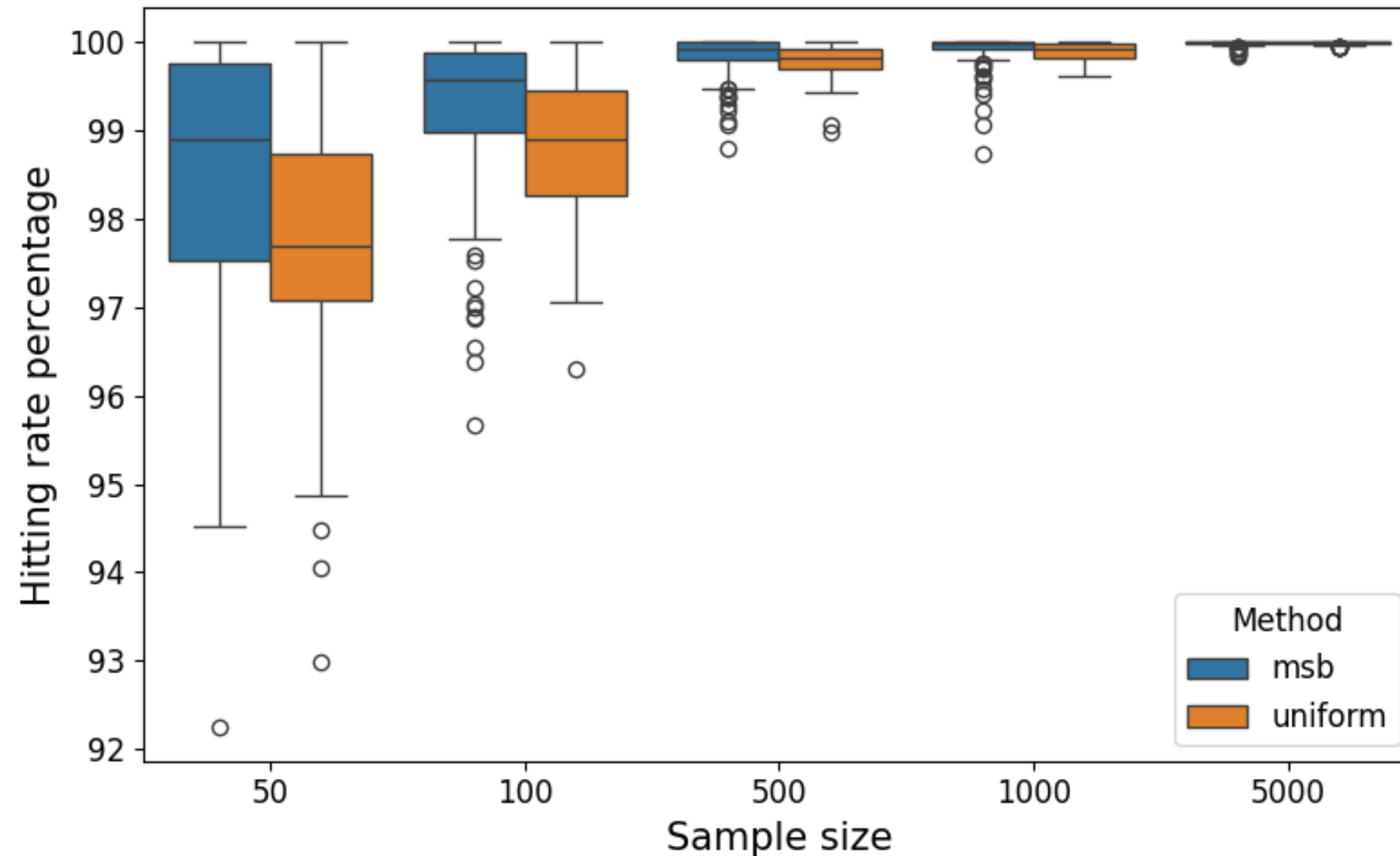


Step 2: for each sample, generate random objective direction, c vector



Step 3: solve an LP with objective vector c and setting the binding constraints to equality

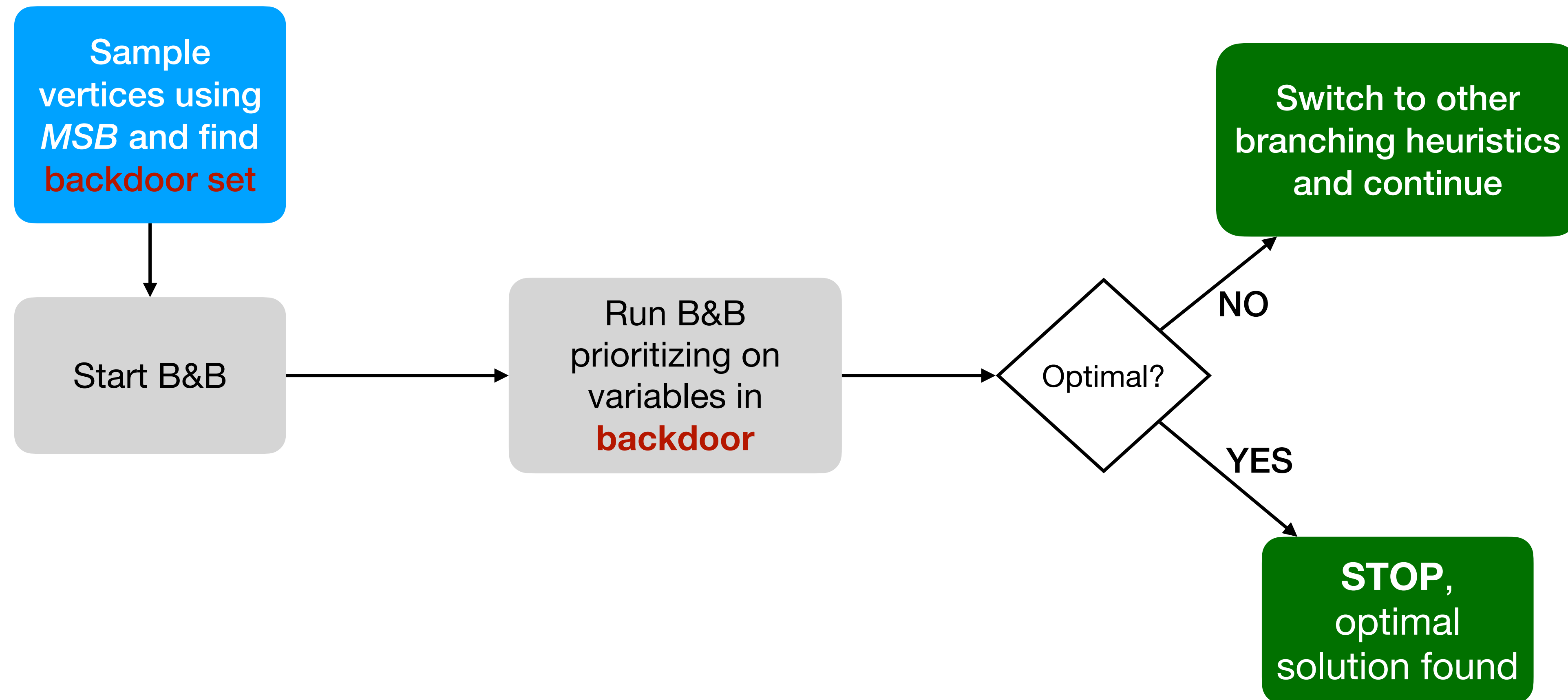
Vertex hitting rate vs. sample size using *MSB*



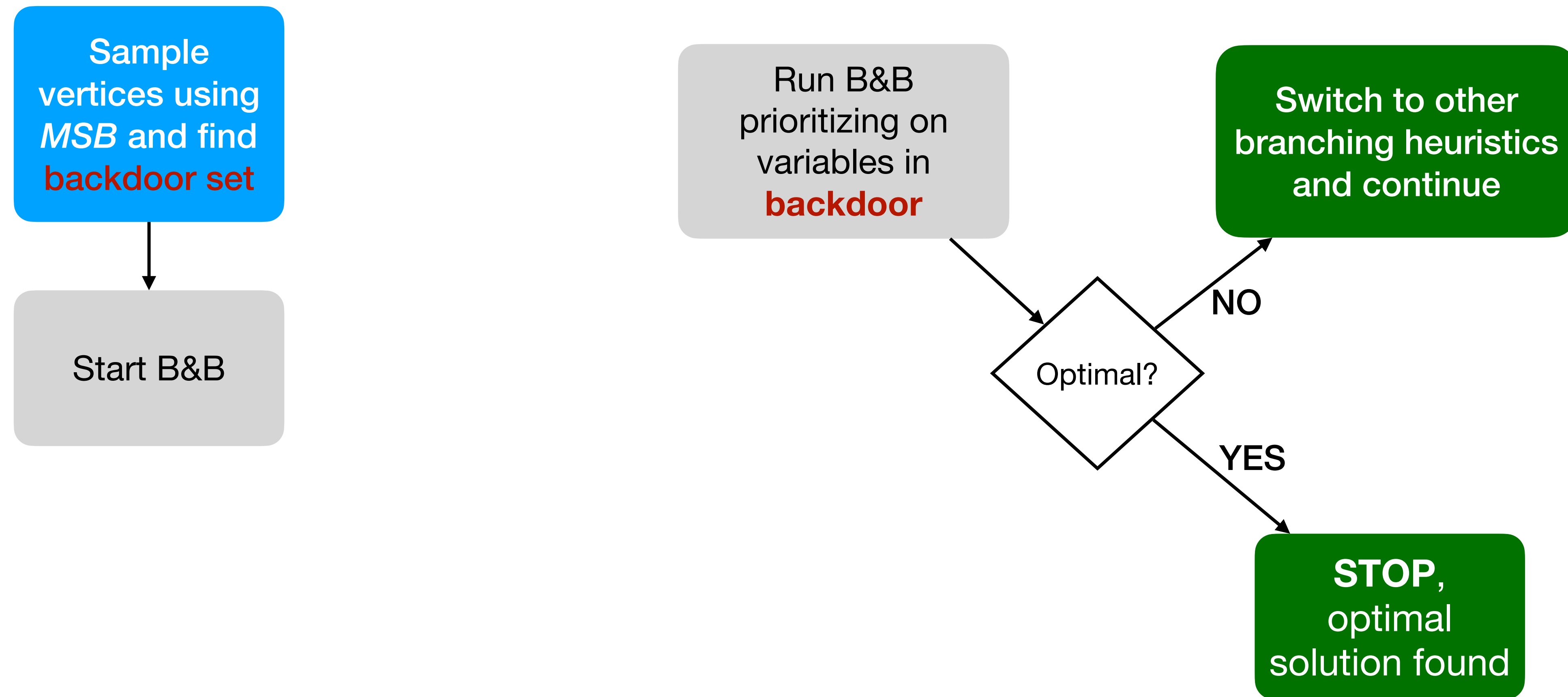
Hitting rate measures how many fractional vertices have been hit by the variables in the

For each of the 100 synthetic instance, we used five different seeds to sample {50,100,500,1000,5000} points using MSB.

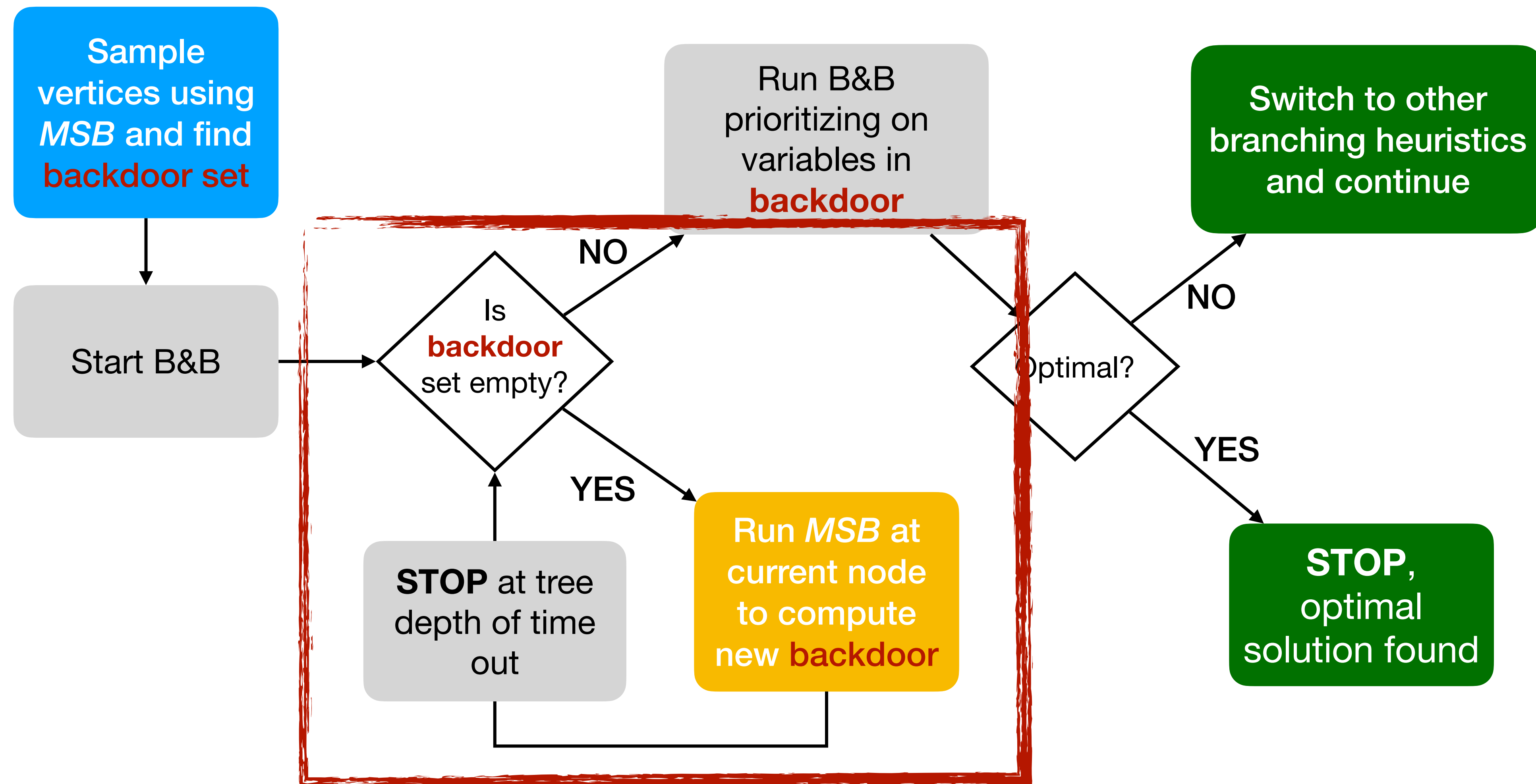
MSB backdoor branching overview



Dynamic MSB backdoor branching



Dynamic MSB backdoor branching



Experiments on MIPLIB17

Acronym	Strategy
S_D	SCIP's default branching
S_MI	SCIP's most infeasible branching
MSB_r (MI)	Root backdoor branching falling back to most infeasible branching
MSB_d (MI)	Dynamic backdoor branching falling back to most infeasible branching
MSB_r (D)	Root backdoor branching falling back to default branching
MSB_d (D)	Root backdoor branching falling back to default branching
FM (D)	Backdoor branching from Fischetti and Monaci with default branching

All instances are solved with **SCIP** version 9.2.2 with Gurobi 12 as the LP solver with presolve, heuristics, cuts, symmetry detection turned off.

Mean nodecount over 5 seeds

Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	13.0	13.0	13.0	13.0	13.0	13.0	13.0
stein15inf	82.2	85.8	105.4	103.4	53.0	103.4	96.6
stein45inf	501.8	1071.4	1031.8	1317.4	440.2	531.0	992.6
p0201	120.2	674.6	1235.2	953.0	444.2	272.2	702.8
glass-sc	249735.8	t.l.	t.l.	t.l.	261353.0	t.l.	t.l.
iis-glass-cov	43613.0	56720.2	54323.0	295638.4	40916.0	174856.0	62862.0
iis-hc-cov	64397.6	125765.0	125765.0	125765.0	64397.6	64397.6	140769.0
supportcase14	5.0	154.0	40.0	37.0	21.0	33.0	53.0
supportcase16	1.0	90.0	39.0	29.0	21.0	29.0	39.0
mod008inf	151.0	1365.0	643.0	627.0	179.0	83.0	961.0
f2gap40400	4.0	696.0	708.0	691.0	9.0	32.0	687.0
mine-166-5	1942.0	t.l.	t.l.	t.l.	239434.0	74421.4	t.l.
opm2-z6-s1	15741.2	t.l.	36245.0	50320.0	1140.2	809.8	t.l.
f2gap401600	4080739.2	4247500.4	t.l.	4285821.6	t.l.	4723332.6	t.l.
acc-tight2	87.2	98.0	195.2	361.2	70.6	579.8	99.0
opm2-z7-s8	36387.2	t.l.	52225.6	t.l.	3779.0	3342.8	t.l.
neos-859770	47172.6	127304.6	99565.0	103057.4	51308.2	40476.2	134863.0
neos-1330346	366712.0	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	49.0	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

* Used the best objective found from MIPLIB as bound for objective function

Mean nodecount over 5 seeds

Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	13.0	13.0	13.0	13.0	13.0	13.0	13.0
stein15inf	82.2	85.8	105.4	103.4	53.0	103.4	96.6
stein45inf	501.8	1071.4	1031.8	1317.4	440.2	531.0	992.6
p0201	120.2	674.6	1235.2	953.0	444.2	272.2	702.8
glass-sc	249735.8	t.l.	t.l.	t.l.	261353.0	t.l.	t.l.
iis-glass-cov	43613.0	56720.2	54323.0	295638.4	40916.0	174856.0	62862.0
iis-hc-cov	64397.6	125765.0	125765.0	125765.0	64397.6	64397.6	140769.0
supportcase14	5.0	154.0	40.0	37.0	21.0	33.0	53.0
supportcase16	1.0	90.0	39.0	29.0	21.0	29.0	39.0
mod008inf	151.0	1365.0	643.0	627.0	179.0	83.0	961.0
f2gap40400	4.0	696.0	708.0	691.0	9.0	32.0	687.0
mine-166-5	1942.0	t.l.	t.l.	t.l.	239434.0	74421.4	t.l.
opm2-z6-s1	15741.2	t.l.	36245.0	50320.0	1140.2	809.8	t.l.
f2gap401600	4080739.2	4247500.4	t.l.	4285821.6	t.l.	4723332.6	t.l.
acc-tight2	87.2	98.0	195.2	361.2	70.6	579.8	99.0
opm2-z7-s8	36387.2	t.l.	52225.6	t.l.	3779.0	3342.8	t.l.
neos-859770	47172.6	127304.6	99565.0	103057.4	51308.2	40476.2	134863.0
neos-1330346	366712.0	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	49.0	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

* Used the best objective found from MIPLIB as bound for objective function

Mean nodecount over 5 seeds

Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	13.0	13.0	13.0	13.0	13.0	13.0	13.0
stein15inf	82.2	85.8	105.4	103.4	53.0	103.4	96.6
stein45inf	501.8	1071.4	1031.8	1317.4	440.2	531.0	992.6
p0201	120.2	674.6	1235.2	953.0	444.2	272.2	702.8
glass-sc	249735.8	t.l.	t.l.	t.l.	261353.0	t.l.	t.l.
iis-glass-cov	43613.0	56720.2	54323.0	295638.4	40916.0	174856.0	62862.0
iis-hc-cov	64397.6	125765.0	125765.0	125765.0	64397.6	64397.6	140769.0
supportcase14	5.0	154.0	40.0	37.0	21.0	33.0	53.0
supportcase16	1.0	90.0	39.0	29.0	21.0	29.0	39.0
mod008inf	151.0	1365.0	643.0	627.0	179.0	83.0	961.0
f2gap40400	4.0	696.0	708.0	691.0	9.0	32.0	687.0
mine-166-5	1942.0	t.l.	t.l.	t.l.	239434.0	74421.4	t.l.
opm2-z6-s1	15741.2	t.l.	36245.0	50320.0	1140.2	809.8	t.l.
f2gap401600	4080739.2	4247500.4	t.l.	4285821.6	t.l.	4723332.6	t.l.
acc-tight2	87.2	98.0	195.2	361.2	70.6	579.8	99.0
opm2-z7-s8	36387.2	t.l.	52225.6	t.l.	3779.0	3342.8	t.l.
neos-859770	47172.6	127304.6	99565.0	103057.4	51308.2	40476.2	134863.0
neos-1330346	366712.0	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	49.0	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

opm2 are precedence constrained knapsack problems with large root LP gap

* Used the best objective found from MIPLIB as bound for objective function

Mean nodecount over 5 seeds

Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	13.0	13.0	13.0	13.0	13.0	13.0	13.0
stein15inf	82.2	85.8	105.4	103.4	53.0	103.4	96.6
stein45inf	501.8	1071.4	1031.8	1317.4	440.2	531.0	992.6
p0201	120.2	674.6	1235.2	953.0	444.2	272.2	702.8
glass-sc	249735.8	t.l.	t.l.	t.l.	261353.0	t.l.	t.l.
iis-glass-cov	43613.0	56720.2	54323.0	295638.4	40916.0	174856.0	62862.0
iis-hc-cov	64397.6	125765.0	125765.0	125765.0	64397.6	64397.6	140769.0
supportcase14	5.0	154.0	40.0	37.0	21.0	33.0	53.0
supportcase16	1.0	90.0	39.0	29.0	21.0	29.0	39.0
mod008inf	151.0	1365.0	643.0	627.0	179.0	83.0	961.0
f2gap40400	4.0	696.0	708.0	691.0	9.0	32.0	687.0
mine-166-5	1942.0	t.l.	t.l.	t.l.	239434.0	74421.4	t.l.
opm2-z6-s1	15741.2	t.l.	36245.0	50320.0	1140.2	809.8	t.l.
f2gap401600	4080739.2	4247500.4	t.l.	4285821.6	t.l.	4723332.6	t.l.
acc-tight2	87.2	98.0	195.2	361.2	70.6	579.8	99.0
opm2-z7-s8	36387.2	t.l.	52225.6	t.l.	3779.0	3342.8	t.l.
neos-859770	47172.6	127304.6	99565.0	103057.4	51308.2	40476.2	134863.0
neos-1330346	366712.0	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	49.0	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

opm2 are precedence constrained knapsack problems with large root LP gap

* Used the best objective found from MIPLIB as bound for objective function

Mean total runtime over 5 seeds

Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	0.51	0.61	1.31	0.76	1.34	1.49	1.57
stein15inf	0.84	0.71	1.29	1.25	1.62	1.10	1.52
stein45inf	1.30	0.90	2.49	7.82	2.97	8.82	1.59
p0201	1.06	0.88	4.47	6.76	4.00	15.83	2.78
glass-sc	3704.05	t.l.	t.l.	t.l.	3967.03	t.l.	t.l.
iis-glass-cov	782.36	769.70	773.66	4665.94	611.92	3010.58	974.62
iis-hc-cov	1964.19	3401.02	3678.02	4003.09	1748.56	1673.75	4050.75
supportcase14	1.37	0.78	3.86	5.20	3.96	5.59	3.81
supportcase16	0.80	1.02	3.78	5.14	4.62	5.09	3.13
mod008inf	1.30	1.05	3.68	4.11	3.66	3.83	3.68
f2gap40400	1.75	1.38	4.83	8.86	5.57	8.87	5.41
mine-166-5	19.94	t.l.	t.l.	t.l.	2273.82	269.48	t.l.
opm2-z6-s1	242.97	t.l.	677.05	1335.62	189.67	175.25	t.l.
f2gap401600	2002.58	2279.28	t.l.	3254.60	t.l.	1280.96	t.l.
acc-tight2	14.92	12.60	127.78	247.86	143.24	207.93	123.39
opm2-z7-s8	993.64	t.l.	2462.81	t.l.	546.10	726.84	t.l.
neos-859770	332.83	905.85	1229.60	2195.76	725.22	1863.86	2106.52
neos-1330346	4999.68	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	31.02	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

Time in seconds

Mean total runtime over 5 seeds

Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	0.51	0.61	1.31	0.76	1.34	1.49	1.57
stein15inf	0.84	0.71	1.29	1.25	1.62	1.10	1.52
stein45inf	1.30	0.90	2.49	7.82	2.97	8.82	1.59
p0201	1.06	0.88	4.47	6.76	4.00	15.83	2.78
glass-sc	3704.05	t.l.	t.l.	t.l.	3967.03	t.l.	t.l.
iis-glass-cov	782.36	769.70	773.66	4665.94	611.92	3010.58	974.62
iis-hc-cov	1964.19	3401.02	3678.02	4003.09	1748.56	1673.75	4050.75
supportcase14	1.37	0.78	3.86	5.20	3.96	5.59	3.81
supportcase16	0.80	1.02	3.78	5.14	4.62	5.09	3.13
mod008inf	1.30	1.05	3.68	4.11	3.66	3.83	3.68
f2gap40400	1.75	1.38	4.83	8.86	5.57	8.87	5.41
mine-166-5	19.94	t.l.	t.l.	t.l.	2273.82	269.48	t.l.
opm2-z6-s1	242.97	t.l.	677.05	1335.62	189.67	175.25	t.l.
f2gap401600	2002.58	2279.28	t.l.	3254.60	t.l.	1280.96	t.l.
acc-tight2	14.92	12.60	127.78	247.86	143.24	207.93	123.39
opm2-z7-s8	993.64	t.l.	2462.81	t.l.	546.10	726.84	t.l.
neos-859770	332.83	905.85	1229.60	2195.76	725.22	1863.86	2106.52
neos-1330346	4999.68	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	31.02	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

Time in seconds

Mean total runtime over 5 seeds

Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	0.51	0.61	1.31	0.76	1.34	1.49	1.57
stein15inf	0.84	0.71	1.29	1.25	1.62	1.10	1.52
stein45inf	1.30	0.90	2.49	7.82	2.97	8.82	1.59
p0201	1.06	0.88	4.47	6.76	4.00	15.83	2.78
glass-sc	3704.05	t.l.	t.l.	t.l.	3967.03	t.l.	t.l.
iis-glass-cov	782.36	769.70	773.66	4665.94	611.92	3010.58	974.62
iis-hc-cov	1964.19	3401.02	3678.02	4003.09	1748.56	1673.75	4050.75
supportcase14	1.37	0.78	3.86	5.20	3.96	5.59	3.81
supportcase16	0.80	1.02	3.78	5.14	4.62	5.09	3.13
mod008inf	1.30	1.05	3.68	4.11	3.66	3.83	3.68
f2gap40400	1.75	1.38	4.83	8.86	5.57	8.87	5.41
mine-166-5	19.94	t.l.	t.l.	t.l.	2273.82	269.48	t.l.
opm2-z6-s1	242.97	t.l.	677.05	1335.62	189.67	175.25	t.l.
f2gap401600	2002.58	2279.28	t.l.	3254.60	t.l.	1280.96	t.l.
acc-tight2	14.92	12.60	127.78	247.86	143.24	207.93	123.39
opm2-z7-s8	993.64	t.l.	2462.81	t.l.	546.10	726.84	t.l.
neos-859770	332.83	905.85	1229.60	2195.76	725.22	1863.86	2106.52
neos-1330346	4999.68	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	31.02	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

Time in seconds

Mean total runtime over 5 seeds

Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	0.51	0.61	1.31	0.76	1.34	1.49	1.57
stein15inf	0.84	0.71	1.29	1.25	1.62	1.10	1.52
stein45inf	1.30	0.90	2.49	7.82	2.97	8.82	1.59
p0201	1.06	0.88	4.47	6.76	4.00	15.83	2.78
glass-sc	3704.05	t.l.	t.l.	t.l.	3967.03	t.l.	t.l.
iis-glass-cov	782.36	769.70	773.66	4665.94	611.92	3010.58	974.62
iis-hc-cov	1964.19	3401.02	3678.02	4003.09	1748.56	1673.75	4050.75
supportcase14	1.37	0.78	3.86	5.20	3.96	5.59	3.81
supportcase16	0.80	1.02	3.78	5.14	4.62	5.09	3.13
mod008inf	1.30	1.05	3.68	4.11	3.66	3.83	3.68
f2gap40400	1.75	1.38	4.83	8.86	5.57	8.87	5.41
mine-166-5	19.94	t.l.	t.l.	t.l.	2273.82	269.48	t.l.
opm2-z6-s1	242.97	t.l.	677.05	1335.62	189.67	175.25	t.l.
f2gap401600	2002.58	2279.28	t.l.	3254.60	t.l.	1280.96	t.l.
acc-tight2	14.92	12.60	127.78	247.86	143.24	207.93	123.39
opm2-z7-s8	993.64	t.l.	2462.81	t.l.	546.10	726.84	t.l.
neos-859770	332.83	905.85	1229.60	2195.76	725.22	1863.86	2106.52
neos-1330346	4999.68	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	31.02	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

Time in seconds

Mean total runtime over 5 seeds

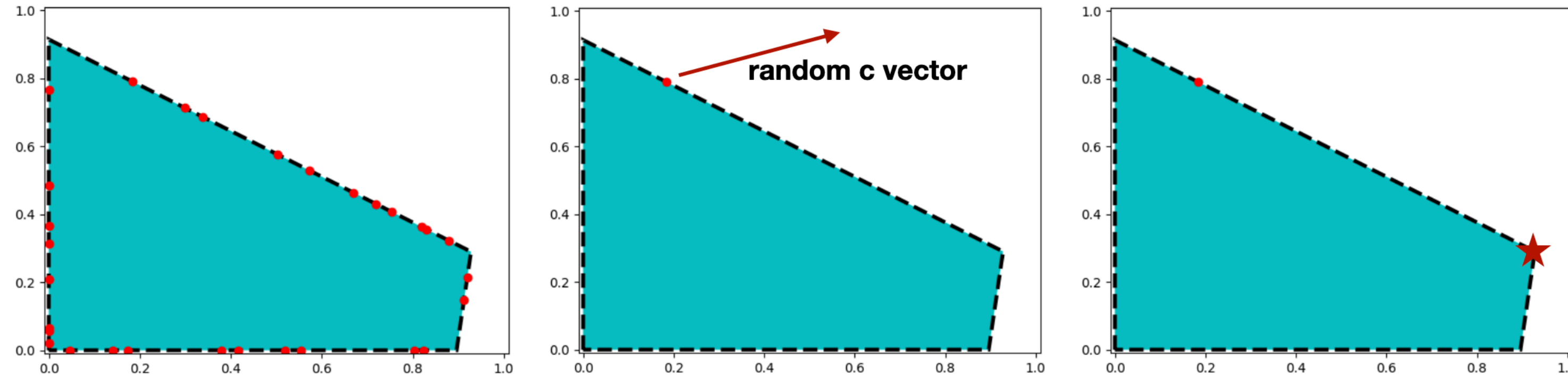
Tested on instances with less than 3000 variables

t.l. : instance reached time limit of 2 hours

Instance	S_D	S_MI	MSB_r (MI)	MSB_d (MI)	MSB_r (D)	MSB_d (D)	FM
stein9inf	0.51	0.61	1.31	0.76	1.34	1.49	1.57
stein15inf	0.84	0.71	1.29	1.25	1.62	1.10	1.52
stein45inf	1.30	0.90	2.49	7.82	2.97	8.82	1.59
p0201	1.06	0.88	4.47	6.76	4.00	15.83	2.78
glass-sc	3704.05	t.l.	t.l.	t.l.	3967.03	t.l.	t.l.
iis-glass-cov	782.36	769.70	773.66	4665.94	611.92	3010.58	974.62
iis-hc-cov	1964.19	3401.02	3678.02	4003.09	1748.56	1673.75	4050.75
supportcase14	1.37	0.78	3.86	5.20	3.96	5.59	3.81
supportcase16	0.80	1.02	3.78	5.14	4.62	5.09	3.13
mod008inf	1.30	1.05	3.68	4.11	3.66	3.83	3.68
f2gap40400	1.75	1.38	4.83	8.86	5.57	8.87	5.41
mine-166-5	19.94	t.l.	t.l.	t.l.	2273.82	269.48	t.l.
opm2-z6-s1	242.97	t.l.	677.05	1335.62	189.67	175.25	t.l.
f2gap401600	2002.58	2279.28	t.l.	3254.60	t.l.	1280.96	t.l.
acc-tight2	14.92	12.60	127.78	247.86	143.24	207.93	123.39
opm2-z7-s8	993.64	t.l.	2462.81	t.l.	546.10	726.84	t.l.
neos-859770	332.83	905.85	1229.60	2195.76	725.22	1863.86	2106.52
neos-1330346	4999.68	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.
toll-like	31.02	t.l.	t.l.	t.l.	t.l.	t.l.	t.l.

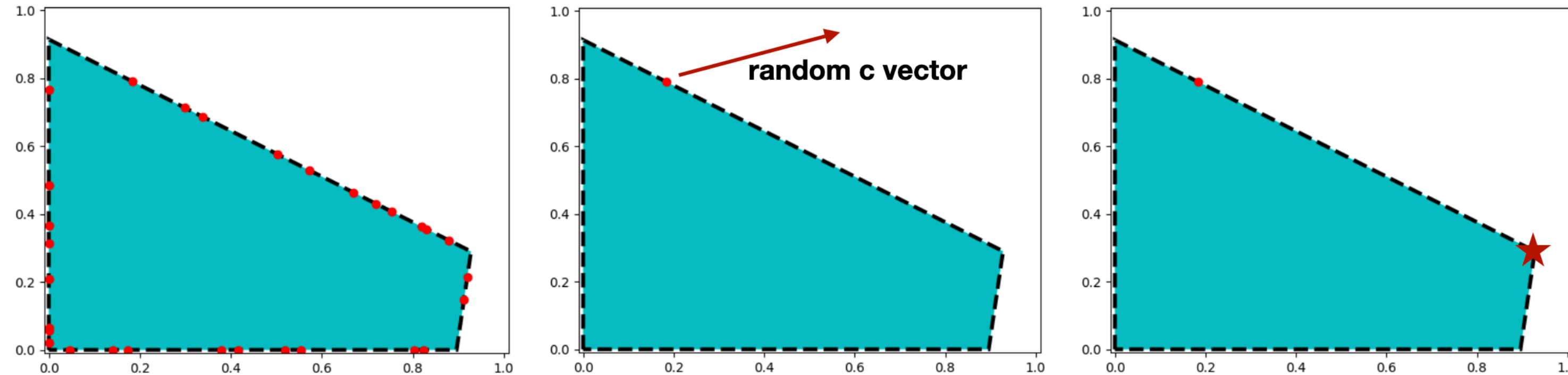
Time in seconds

Conclusions



- We propose a dynamic sampling based approach to search for backdoors in BLP
- We observe existence of small backdoors on small generated instances
- MSB provides a good approximation for uniform vertex sampling
- MSB_r and MSB_d outperforms SCIP's default branching in 10/19 MIPLIB instances
- Branching on “good” backdoor variables at the top of the BnB tree has positive impact

Conclusions



- We propose a dynamic sampling based approach to search for backdoors in BLP
- We observe existence of small backdoors on small generated instances
- MSB provides a good approximation for uniform vertex sampling
- MSB_r and MSB_d outperforms SCIP's default branching in 10/19 MIPLIB instances
- Branching on “good” backdoor variables at the top of the BnB tree has positive impact

Future work

Supervised learning based on vertex coverage of variables sets, diffusion models, energy-based sampling

Thank you!

Questions?

changy.liu@mail.utoronto.ca

<https://changyliu.github.io/>

Appendices

Strong Optimality Backdoor

$$(P) \quad \min_x \mathbf{c}^T x \text{ s.t. } Ax \leq \mathbf{b}, x \in \{0,1\}^n, n \gg 3$$

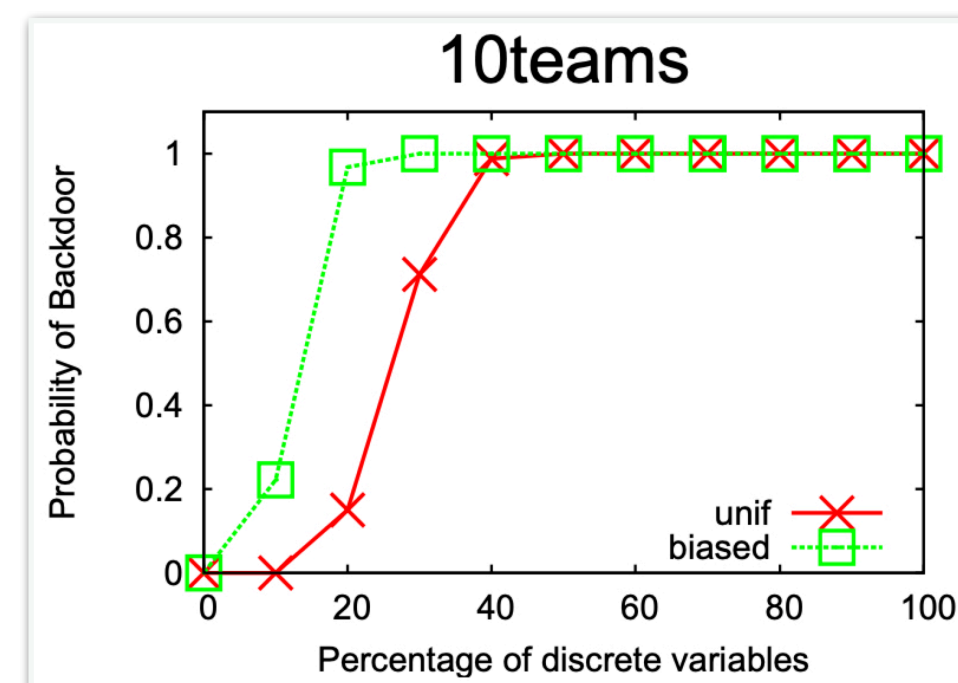
- Let $v(P)$ denote the optimal value of the problem (P) or inf if the problem is infeasible.
- Let $L(S) := \min\{c^T x : Ax \leq b, x_j \text{ integer } \forall j \in S\}$ be the optimal value obtained by only requiring integrality for variables in set S . $L(S) := inf$ if the problem is infeasible by requiring integrality on variables in S .
- As a result, $L(\emptyset)$ corresponds to the root linear relaxation value resulting from dropping the integrality requirement on all variables in (P) .

Definition: A set S is a **strong optimality backdoor** iff $L(S) = v(P)$.

Existing Methods

Variable Sampling

1. Pick random subsets of variables, B , of size k
 - account for “fractionality” of the variables in the LP root relaxation
2. Force MIP solver to **branch on B** only
3. B is a **backdoor** if instance is **solved**



Dilkina et al. “Backdoors to combinatorial optimization: Feasibility and optimality”. CPAIOR 2009

Monte-Carlo Tree Search

Search space:

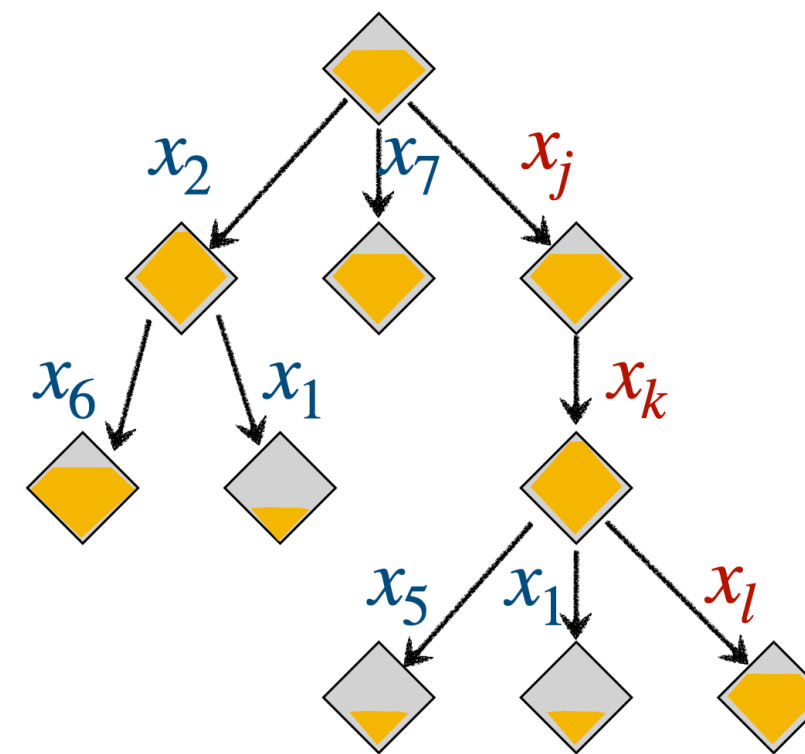
set of all possible size- K permutations of the variables

Branching factor:

number of integer vars.

Reward estimate:

how “good” are the backdoors I’ve seen in this node’s subtree?



Khalil et al., “Finding backdoors to integer programs: a Monte Carlo tree search framework”, AAAI 2022

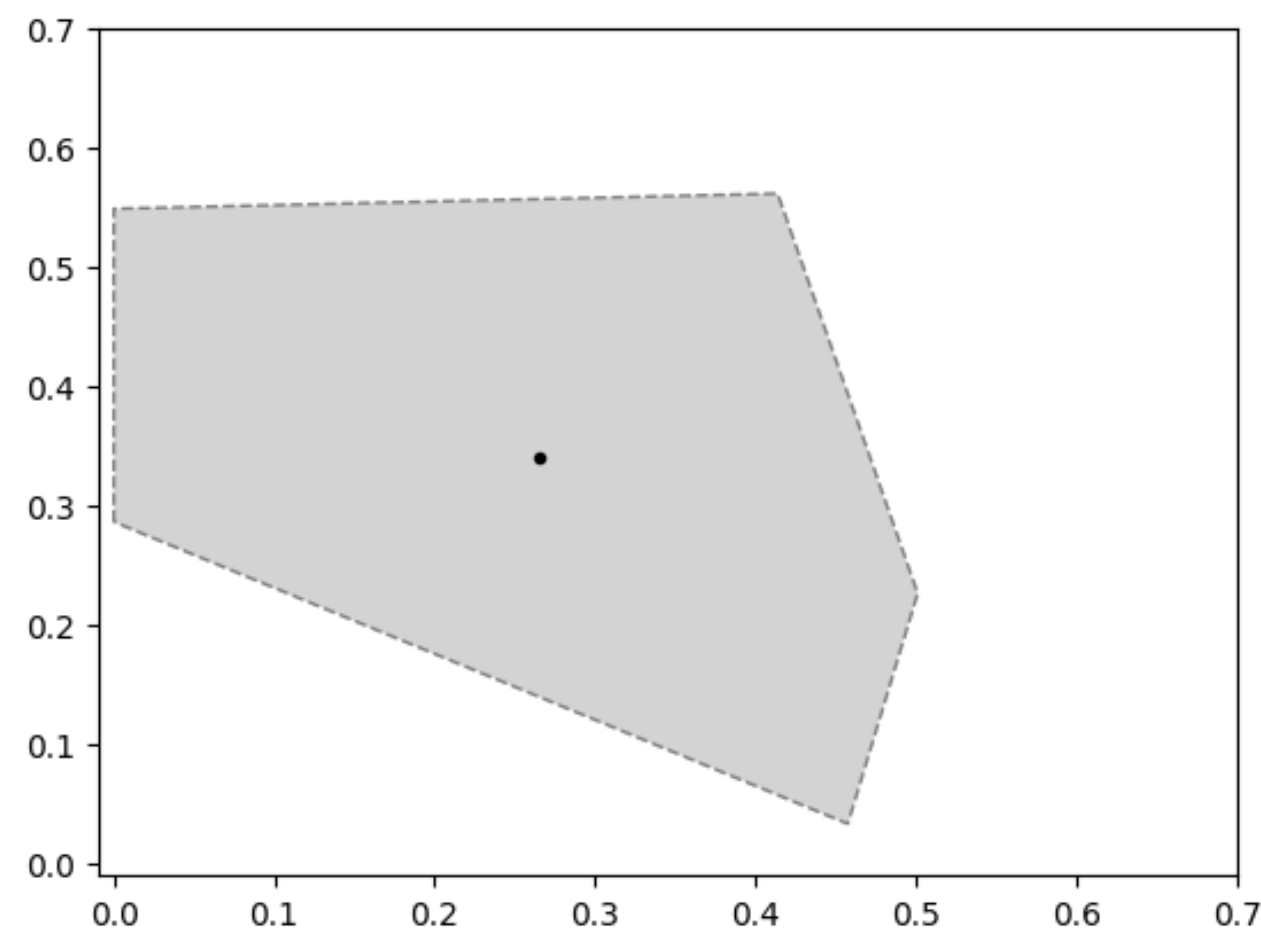
Learning-based Methods

1. Trains NN to predict whether a candidate set B is a good pseudo-backdoor by evaluating randomly sampled backdoor on known MIP instances.
2. Can also use MCTS to collect higher-quality backdoor candidates.

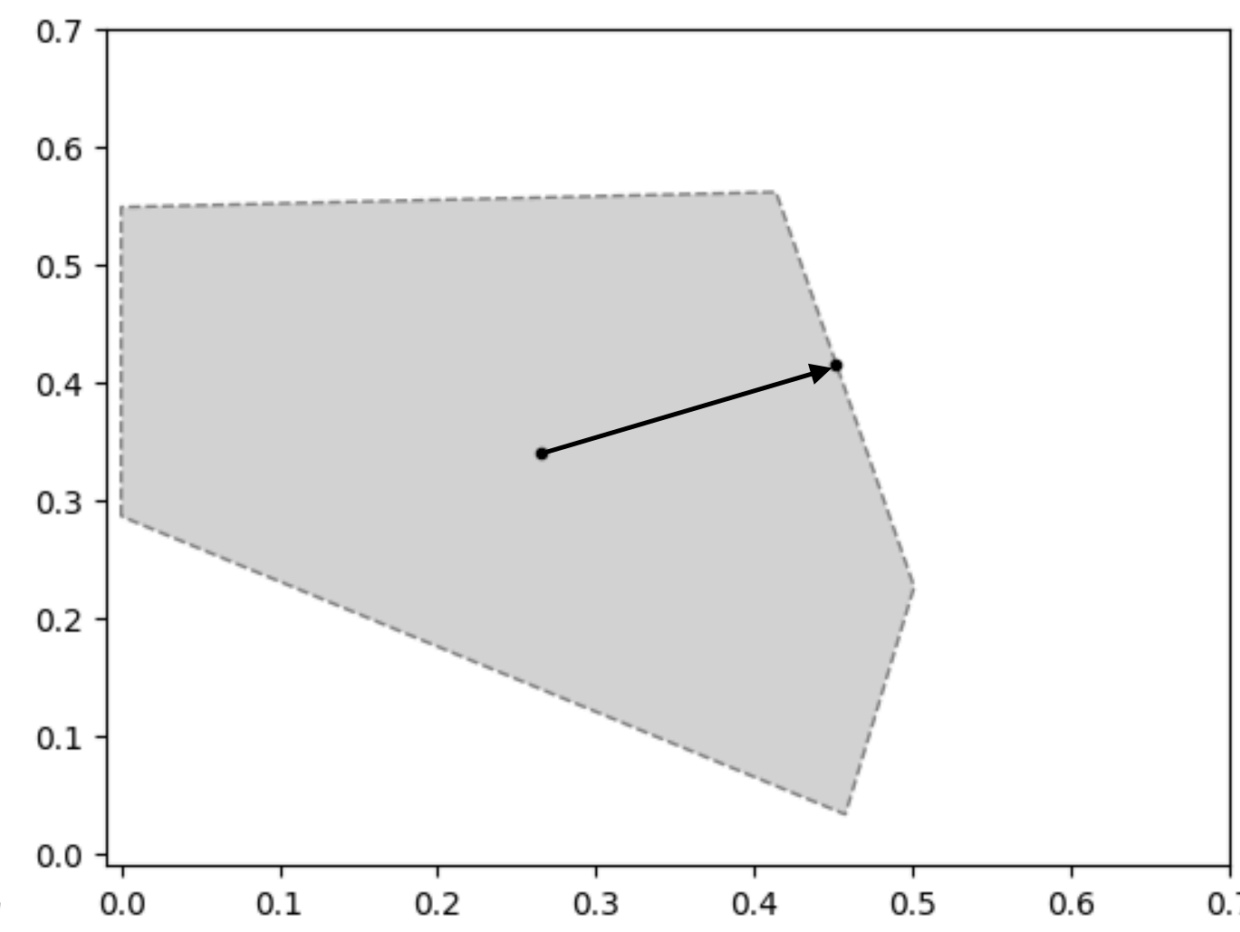
Ferber et al. “Learning pseudo-backdoors for mixed integer programs.” CPAIOR 2022.

Cai et al. “Learning Backdoors for Mixed Integer Linear Programs with Contrastive Learning.” ECAI 2024

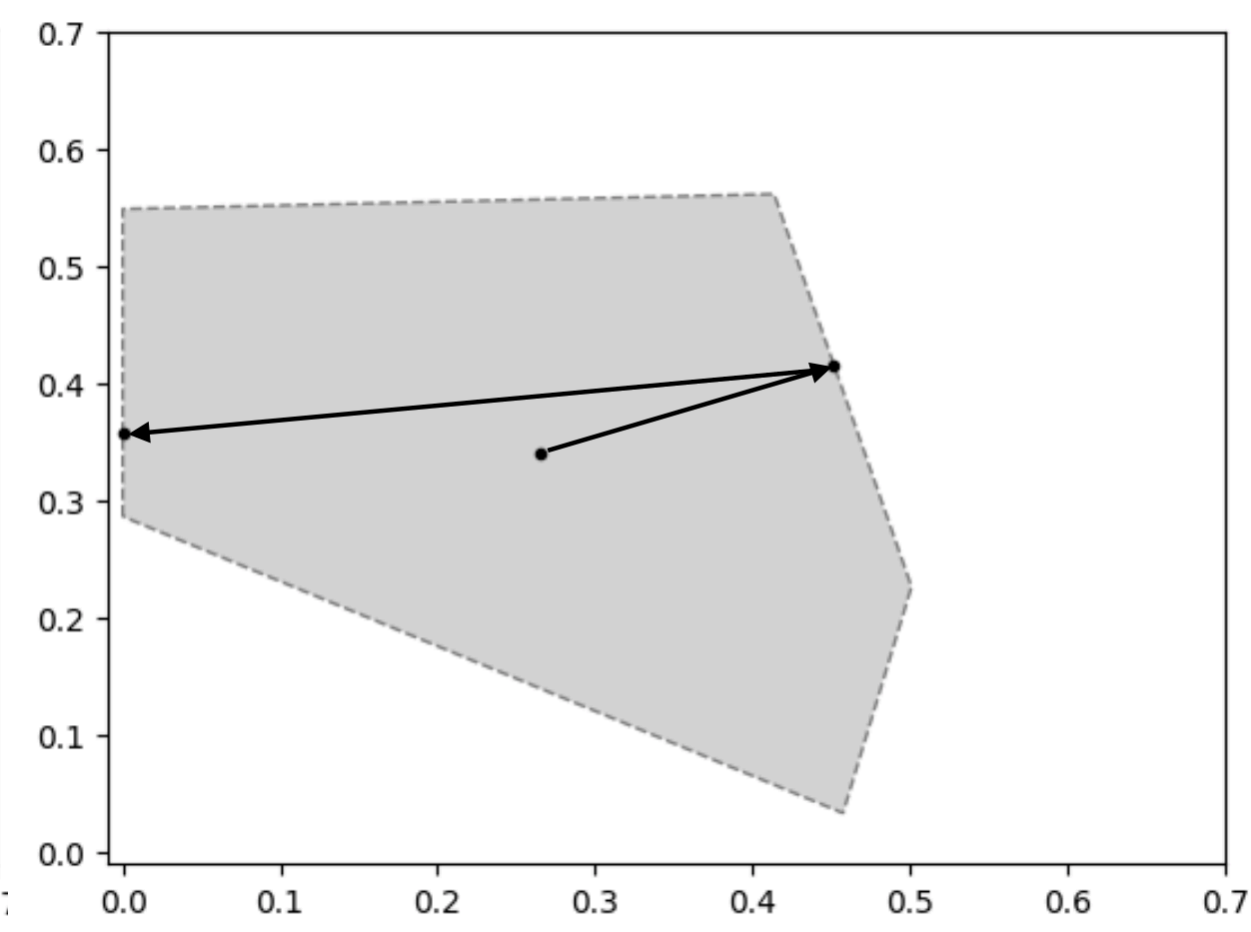
Shake-and-bake (SB)



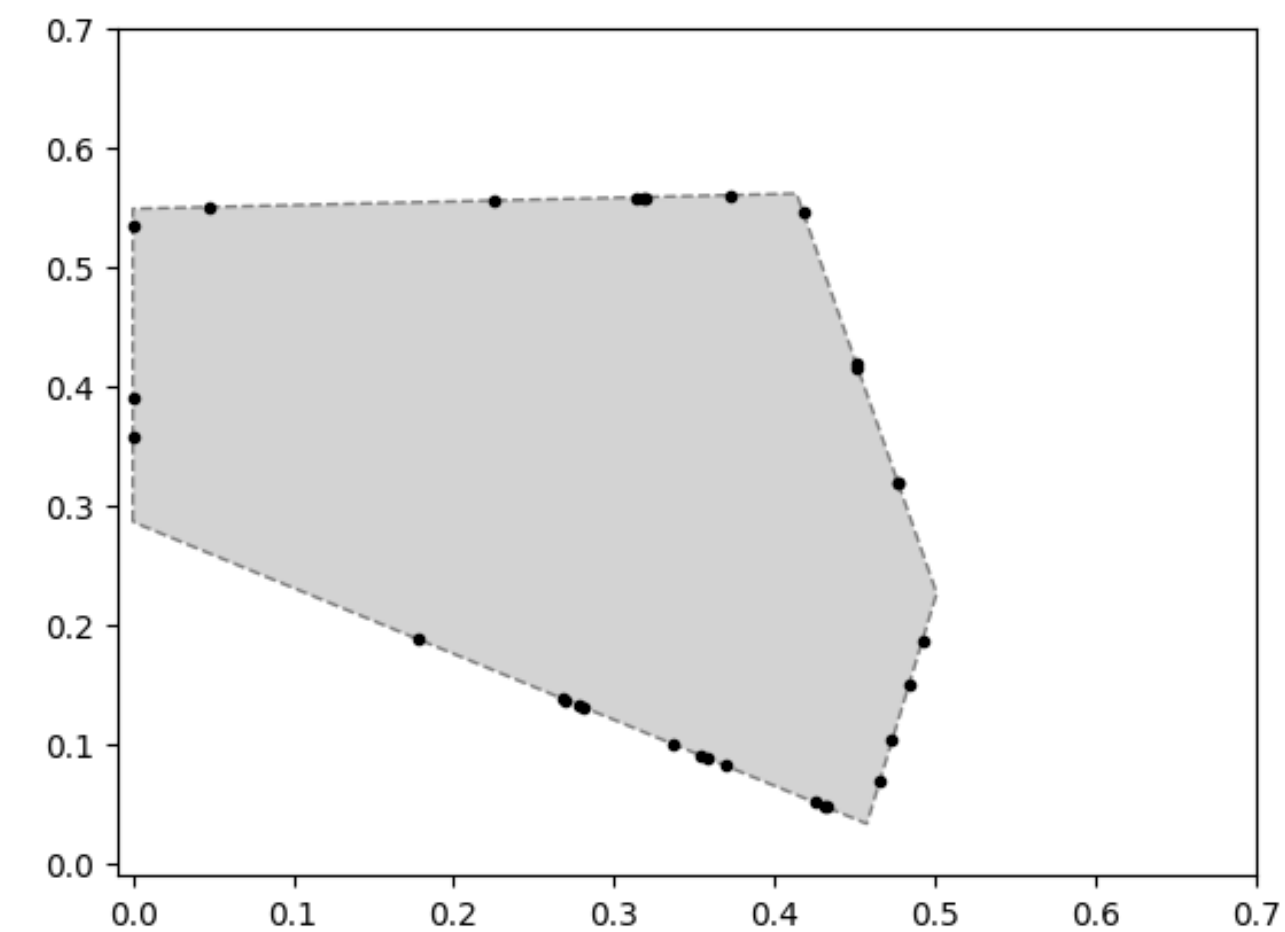
(a) Find Chebyshev centre of $\mathcal{P}$



(b) Findpoint x^0 on boundary of $\mathcal{P}$



(c) Generate direction from x^0 's binding constraint and hits point x^1



(d) 30 samples on boundary of $\mathcal{P}$

Boender et al. “*Shake-and-bake algorithms for generating uniform points on the boundary of bounded polyhedra*”. Operations research, 1991.

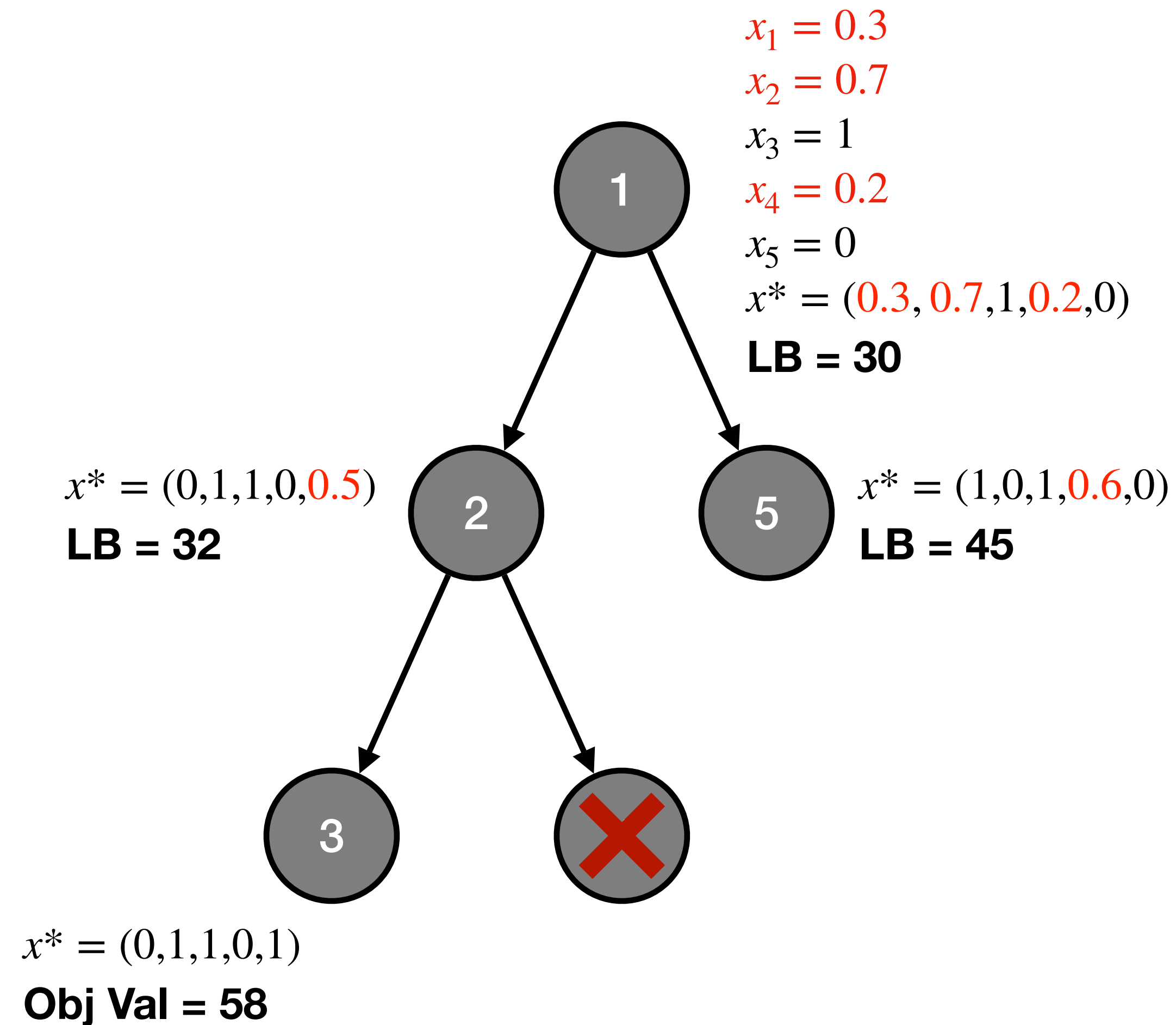
Set covering for binary linear program

0. Initialization

- Let set of backdoor variables $S := \emptyset$
- Let set of fractional solutions $F := \emptyset$
- Preset upper bound on number of fractional solutions found to stop branch and bound $K = 3$

Set covering for binary linear program

1. Run branch and bound until $K = 3$ fractional solutions found



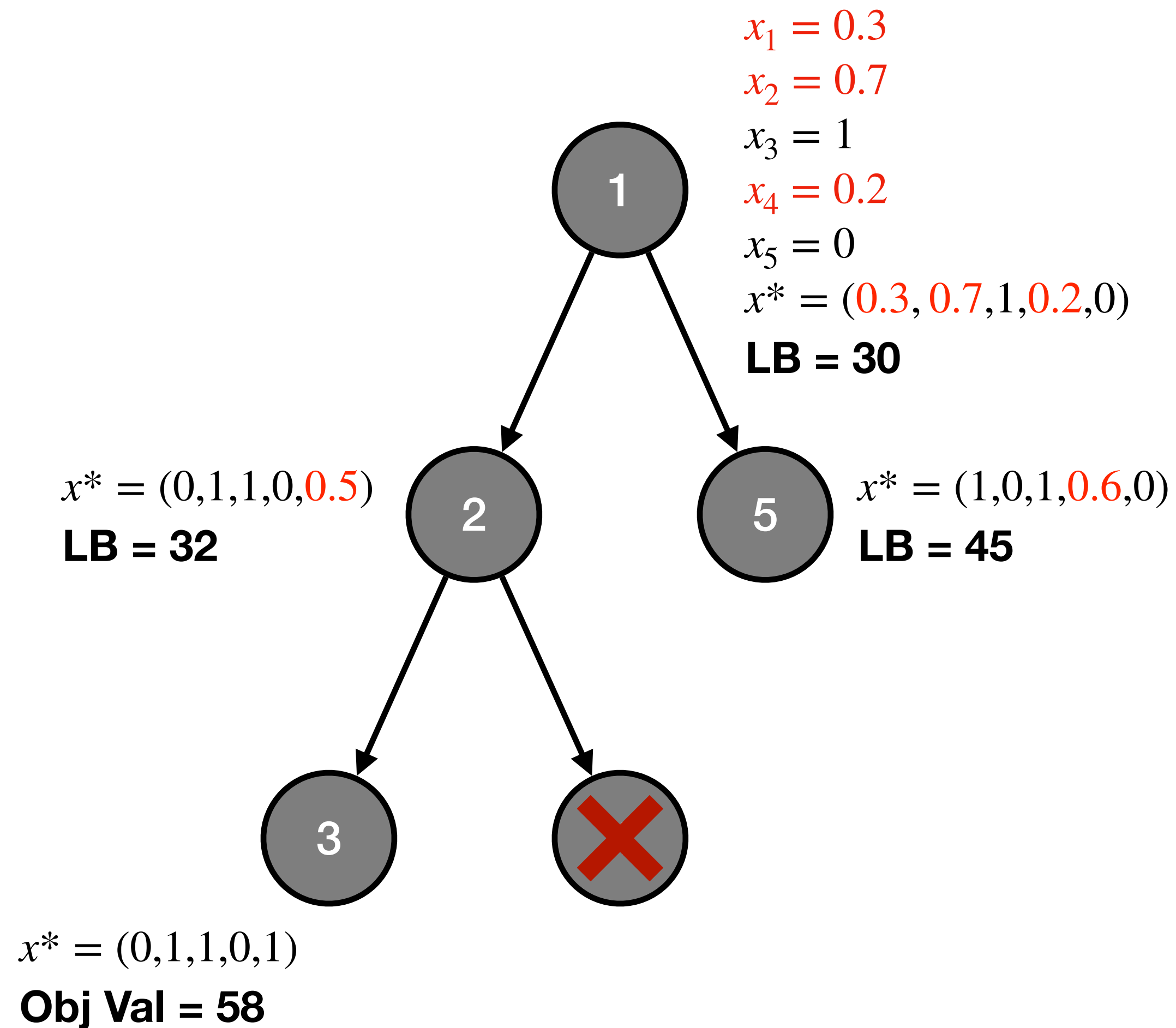
$$F = \left\{ \begin{pmatrix} 0.3 & 0.7 & 1 & 0.2 & 0 \end{pmatrix}, \begin{pmatrix} 0 & 1 & 1 & 0 & 0.5 \end{pmatrix}, \begin{pmatrix} 1 & 0 & 1 & 0.6 & 0 \end{pmatrix} \right\}$$

Set covering for binary linear program

2. Solve a set covering problem

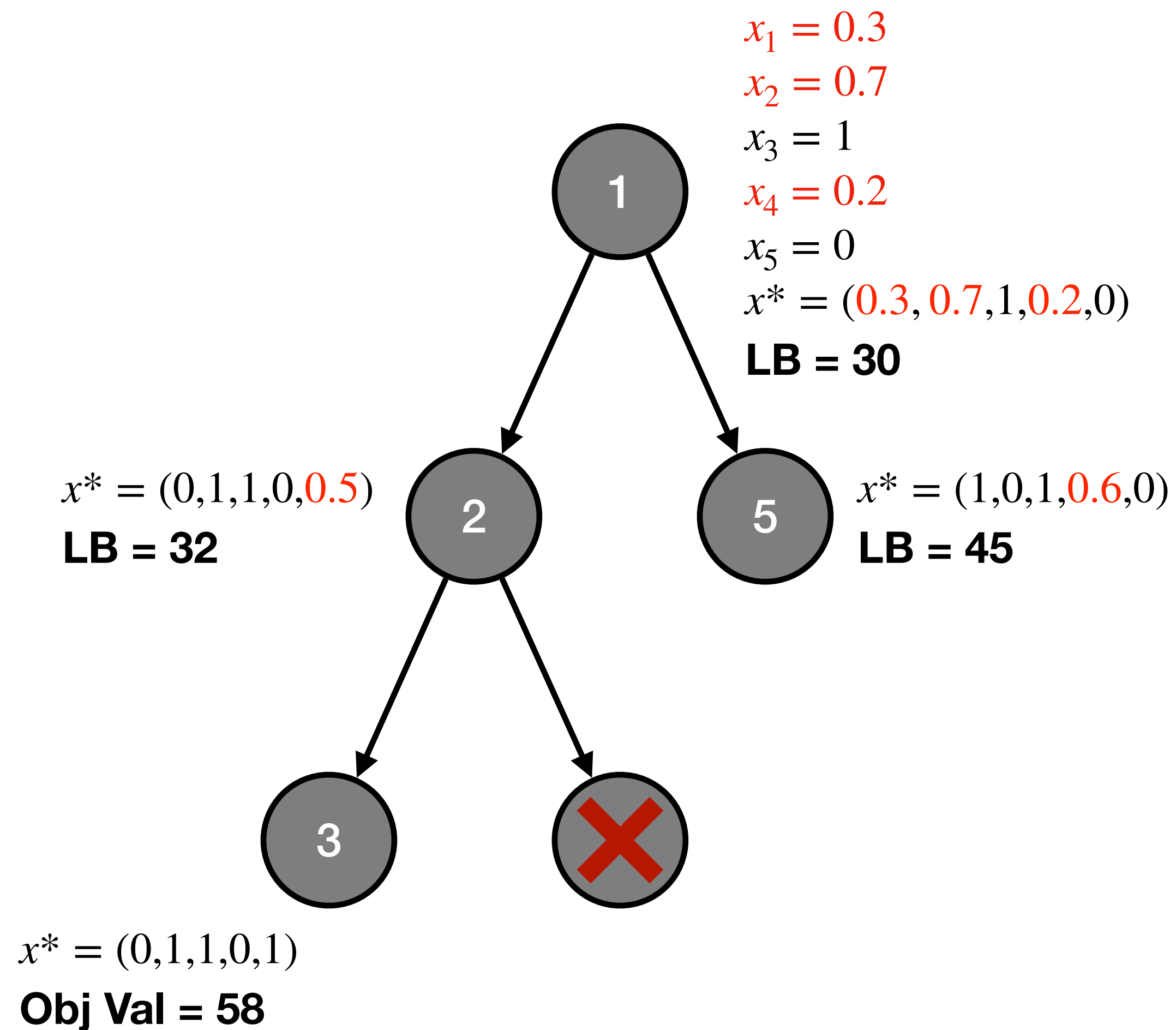
Let $\mathcal{J}$ be the set of all binary integer variables
 For each $j \in \mathcal{J}$, let $y_j = 1$ if x_j in S and 0, o.w.

$$\begin{aligned} \min \quad & \sum_{j \in \mathcal{J}} y_j \\ \text{s.t.} \quad & \sum_{j \in \text{frac}(x^*)} y_j \geq 1, \quad \forall x^* \in F_k \\ & y_j \in \{0,1\}, \quad \forall j \in \mathcal{J} \end{aligned}$$



Set covering for binary linear program

2. Solve a set covering problem

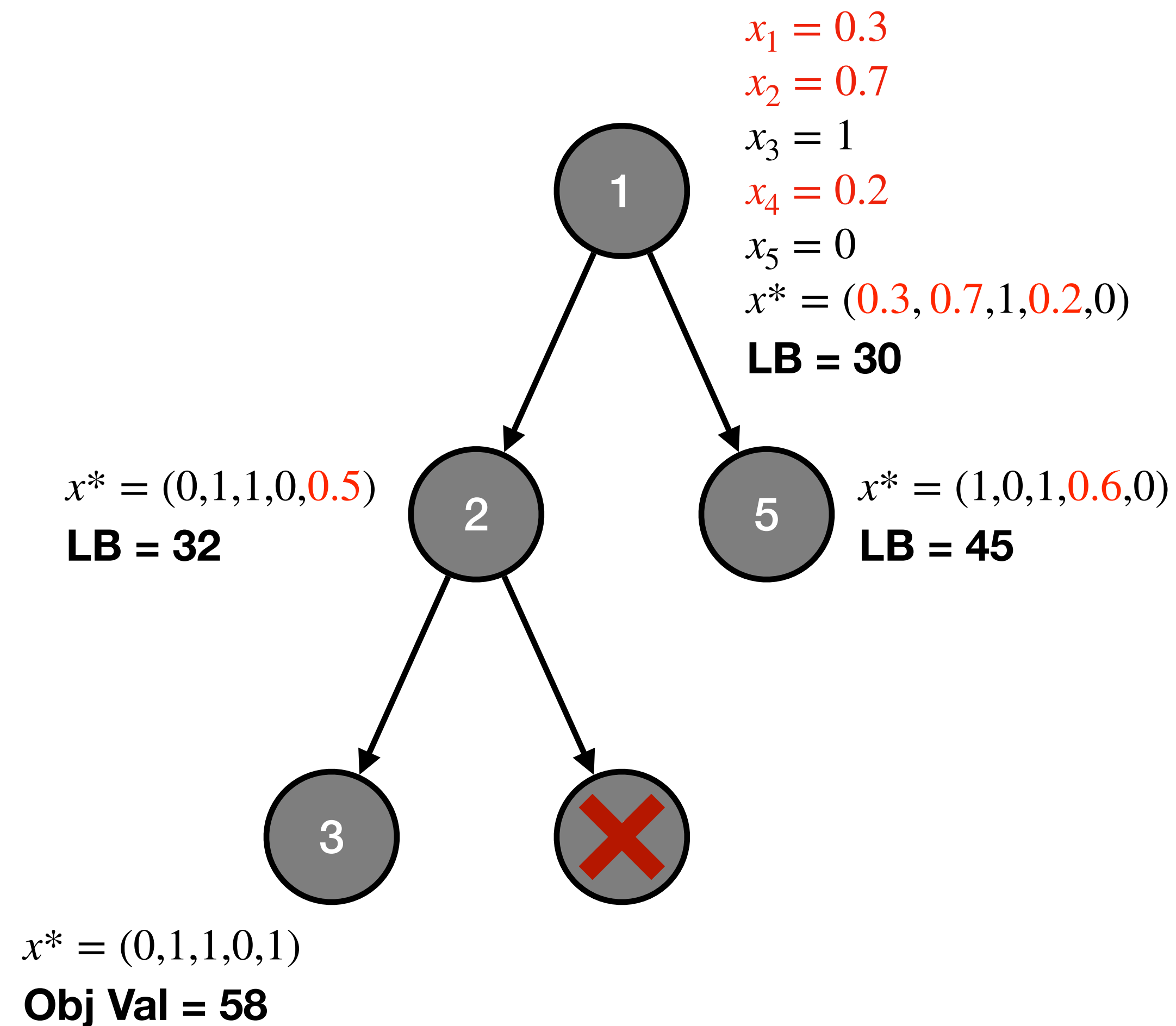


$$\begin{aligned}
 &\min \sum_{j \in \mathcal{J}} y_j \\
 &\text{s.t.} \quad \sum_{j \in \text{frac}(x^*)} y_j \geq 1, \quad \forall x^* \in F_k \\
 &\quad y_j \in \{0, 1\}, \quad \forall j \in \mathcal{J}
 \end{aligned}$$

$$F_k = \left\{ \begin{pmatrix} 0.3, & 0.7, & 1, & 0.2, & 0 \end{pmatrix}, \begin{pmatrix} 0, & 1, & 1, & 0, & 0.5 \end{pmatrix}, \begin{pmatrix} 1, & 0, & 1, & 0.6, & 0 \end{pmatrix} \right\}$$

Set covering for binary linear program

2. Solve a set covering problem



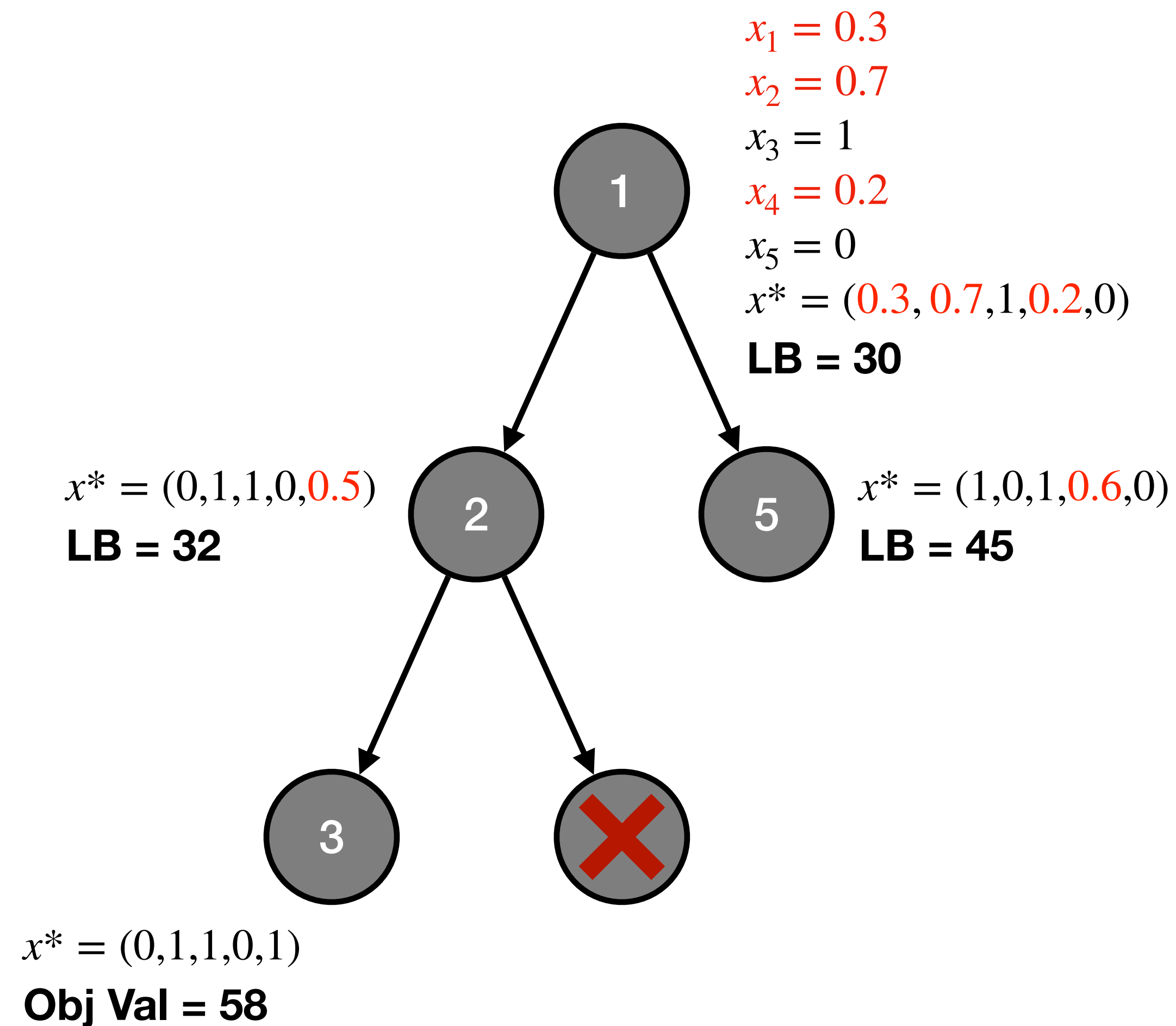
$$\begin{aligned}
 &\min \sum_{j \in \mathcal{J}} y_j \\
 &\text{s.t.} \quad \sum_{j \in \text{frac}(x^*)} y_j \geq 1, \quad \forall x^* \in F_k \\
 &\quad y_j \in \{0, 1\}, \quad \forall j \in \mathcal{J}
 \end{aligned}$$

$$F_k = \left\{ \begin{pmatrix} 0.3, & 0.7, & 1, & 0.2, & 0 \end{pmatrix}, \begin{pmatrix} 0, & 1, & 1, & 0, & 0.5 \end{pmatrix}, \begin{pmatrix} 1, & 0, & 1, & 0.6, & 0 \end{pmatrix} \right\}$$

$\uparrow$ $\uparrow$
 x_4 x_5

Set covering for binary linear program

2. Solve a set covering problem



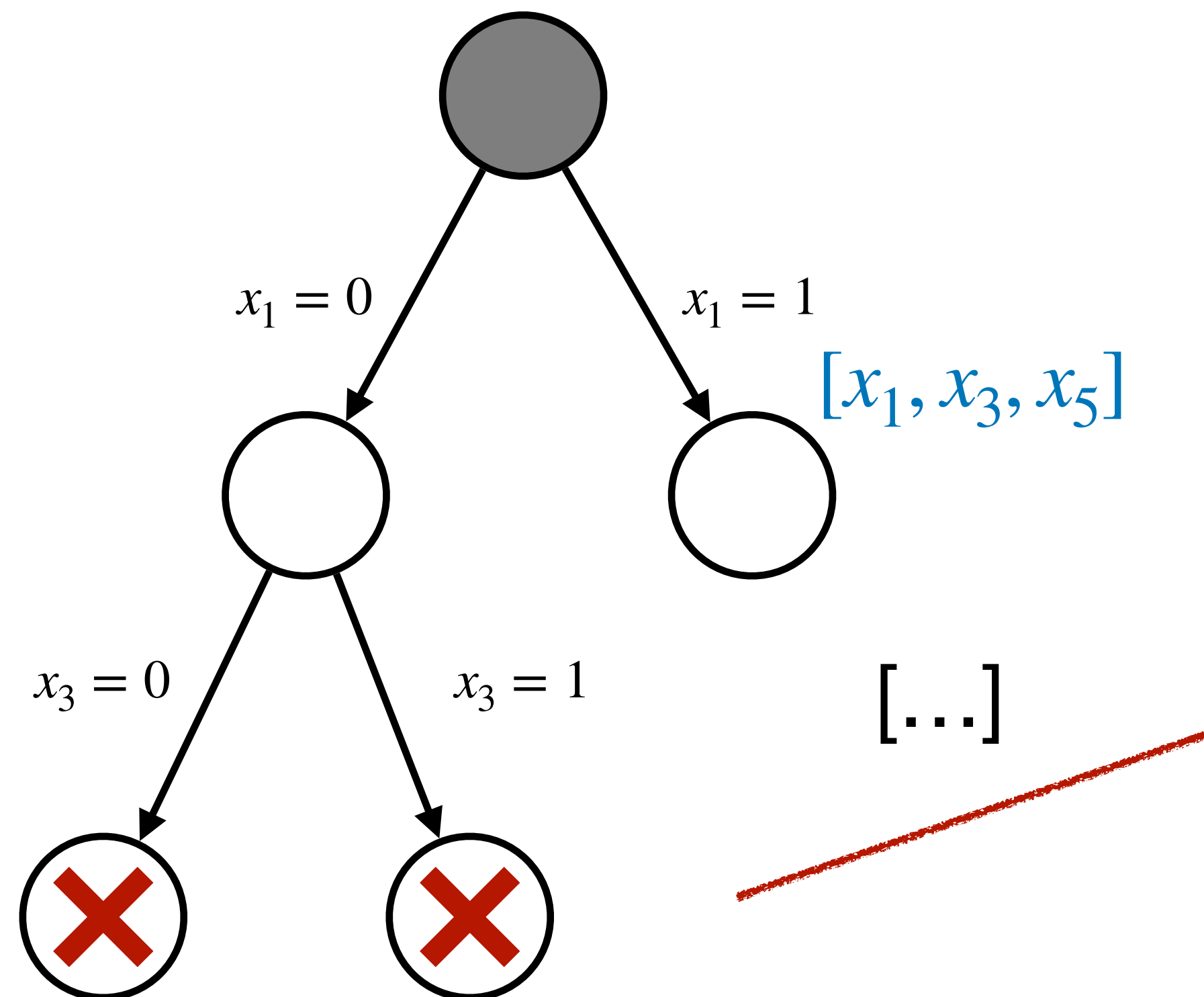
$$\begin{aligned}
 &\min \sum_{j \in \mathcal{J}} y_j \\
 &\text{s.t.} \quad \sum_{j \in \text{frac}(x^*)} y_j \geq 1, \quad \forall x^* \in F_k \\
 &\quad y_j \in \{0, 1\}, \quad \forall j \in \mathcal{J}
 \end{aligned}$$

$$y^* = (0, 0, 0, 1, 1)$$

$$\text{Set } S = \{x_4, x_5\}$$

Dynamic backdoor branching

Quasi-backdoor: $[x_1, x_3]$



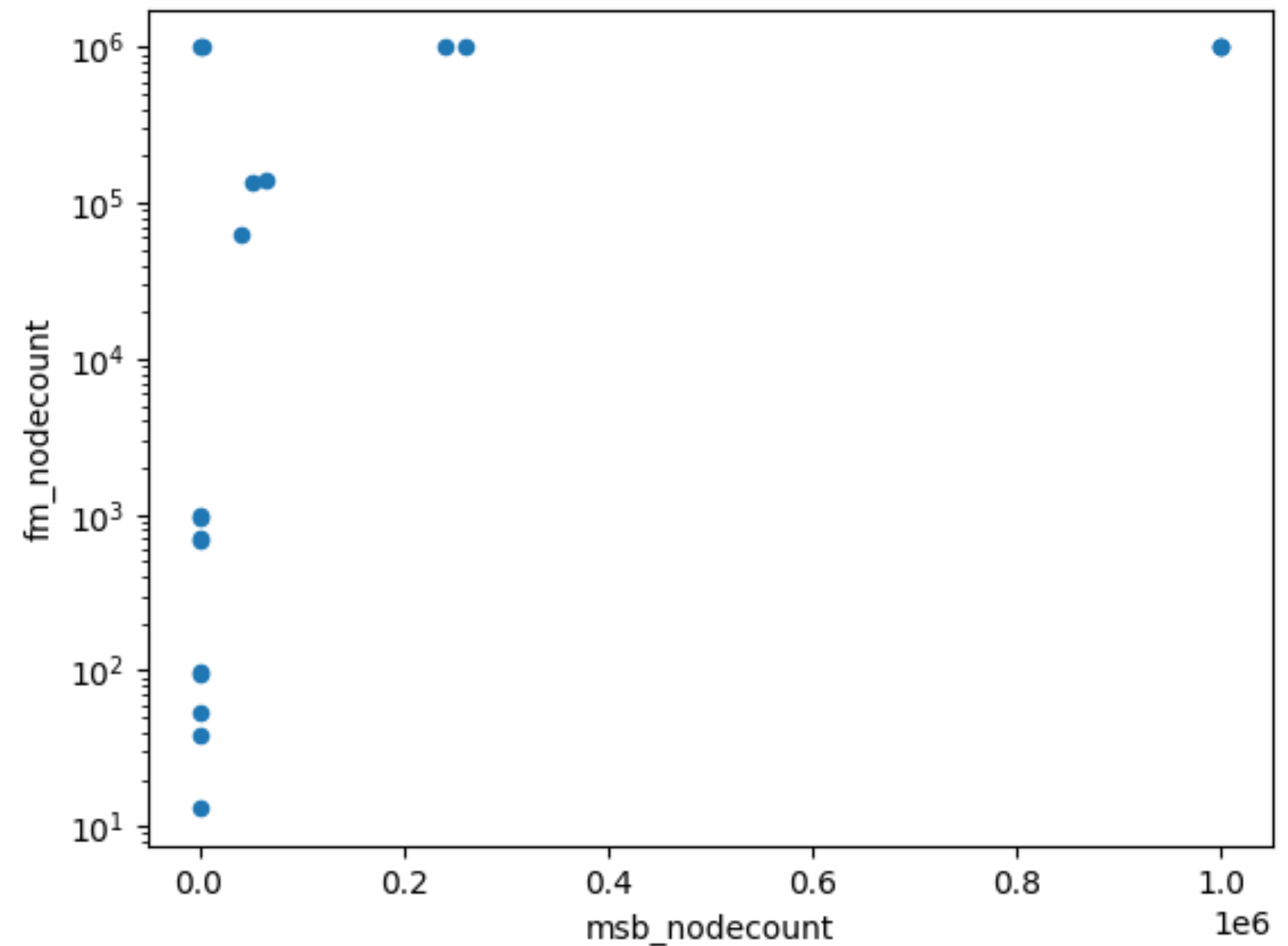
1. Sample K vertices using MSB and obtain a quasi-backdoor.
2. Branch by prioritizing the variables in the quasi-backdoor.
3. At any node where no branching candidate is in the quasi-backdoor.
 1. Run MSB at current node by adding previous branching decisions to generate new quasi-backdoor.
 2. Merge backdoor variables from parents to current node
 3. Return to step 2.
4. If max tree_depth d is reached, fall back to most infeasible or default branching.
5. If problem is solved or time limit is hit, stop.

Dynamic MSB hyper-parameters

Parameter	Value
Initial number of samples (K) at root	500
Initial number of samples in the BnB tree	100
Sampling decay factor (as a function of node depth)	0.8
Sampling max depth (d)	8
Minimum number of samples	20
Maximum sampling time at root	1800
Maximum sampling time in the BnB tree	1800
Total maximum runtime	7200

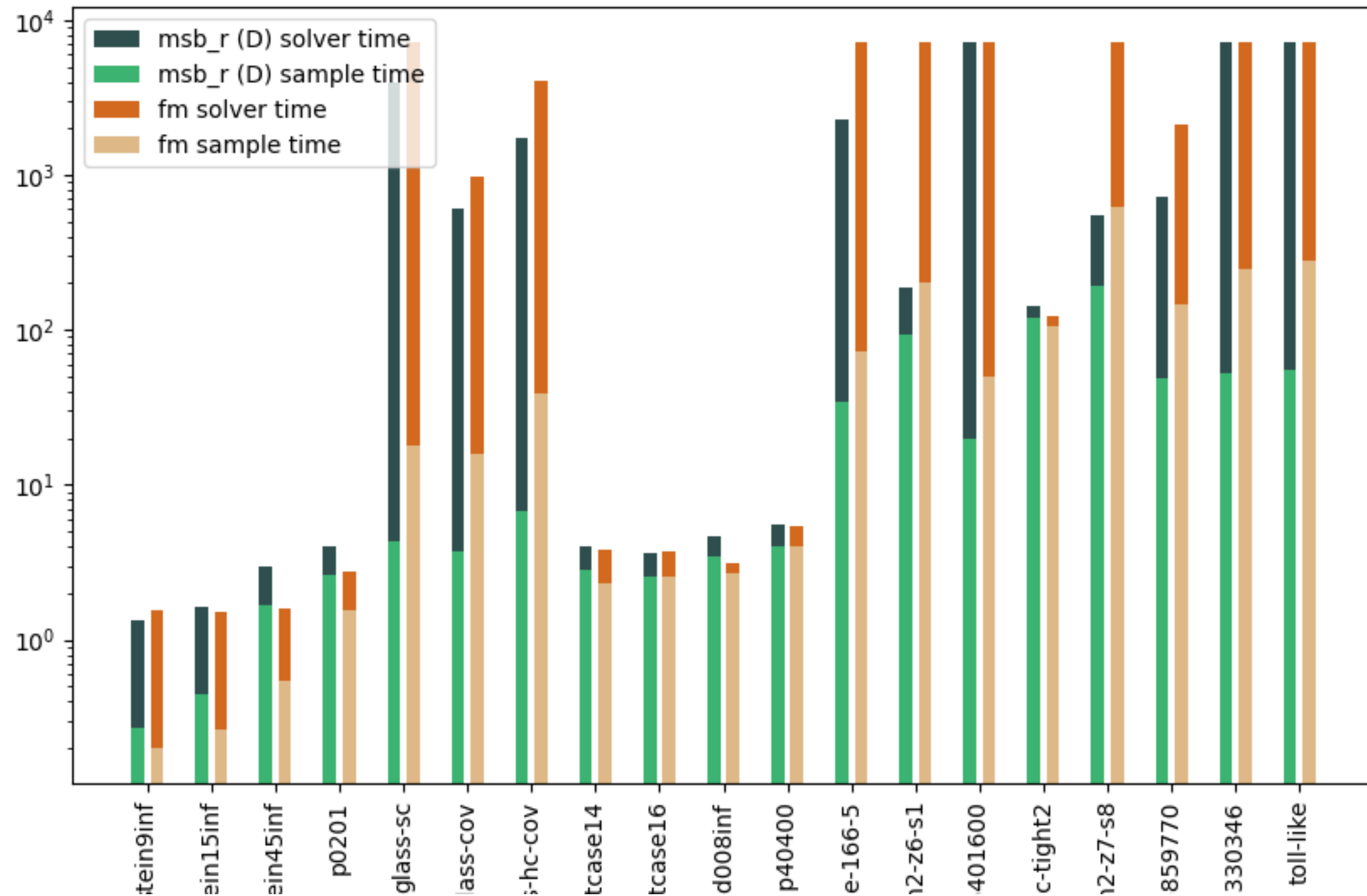
MSB_r (D) vs FM nodecount

Instance	MSB_r (D)	FM
stein9inf	13.0	13.0
stein15inf	53.0	96.6
stein45inf	440.2	992.6
p0201	444.2	702.8
glass-sc	261353.0	t.l.
iis-glass-cov	40916.0	62862.0
iis-hc-cov	64397.6	140769.0
supportcase14	21.0	53.0
supportcase16	21.0	39.0
mod008inf	179.0	961.0
f2gap40400	9.0	687.0
mine-166-5	239434.0	t.l.
opm2-z6-s1	1140.2	t.l.
f2gap401600	t.l.	t.l.
acc-tight2	70.6	99.0
opm2-z7-s8	3779	t.l.
neos-859770	51308.2	134863.0
neos-1330346	t.l.	t.l.
toll-like	t.l.	t.l.



Axis are logscale

MSB_r (D) vs FM runtime



y-axis is logscale